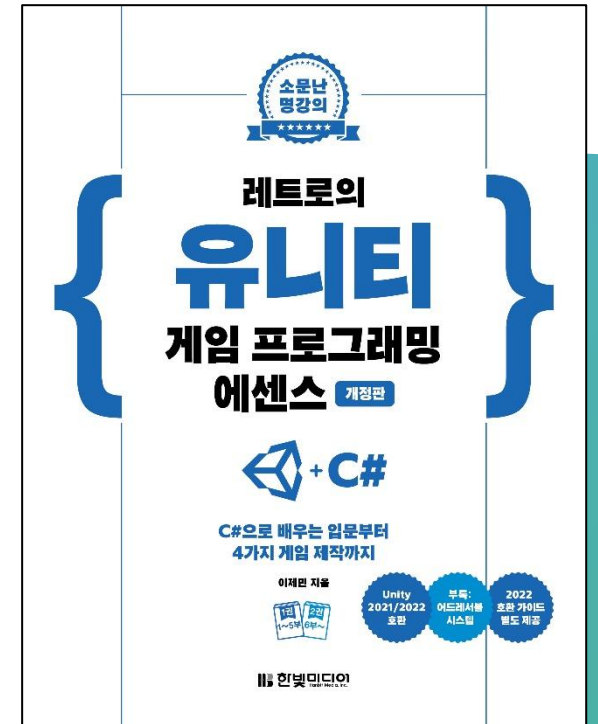


▶ Chapter 15: 좀비 서바이버 총과 슈터

레트로의 유니티 게임프로그래밍 에센스 (개정판)



이제만 지음

Contents

- Chapter 15 좀비 서바이버 총과 슈터
 - 15.1 인터페이스
 - 15.2 총 게임 오브젝트 준비
 - 15.3 GunData 스크립트
 - 15.4 Gun 스크립트
 - 15.5 슈터 만들기
 - 15.6 마치며



CHAPTER 15 좀비 서바이버 총과 슈터

- C# 인터페이스를 사용한 '느슨한 커플링'
- 슈터 게임의 총 제작 방법
- 라인 렌더러를 사용해 광선 그리기
- 레이캐스트를 사용해 탄알 발사 구현하기
- 코루틴을 사용해 대기 시간 삽입하기
- IK를 사용해 총을 잡도록 애니메이션 변경하기

15.1 인터페이스(1)

15.1.1 C# 인터페이스

- C# 인터페이스는 어떤 메서드를 구현하도록 강제하는 계약
 - 인터페이스를 상속하는 클래스는 해당 인터페이스의 메서드를 반드시 구현
- IItem은 아이템 클래스들이 상속하는 인터페이스(인터페이스는 이름 앞에 I를 붙여 선언하는 것이 관례)

```
public interface IItem {  
    void Use(GameObject target);  
}
```

15.1 인터페이스(2)

- IItem을 상속한 클래스는 Use() 메서드를 반드시 구현
 - 따라서 AmmoPack 클래스와 HealthPack 클래스는 Use() 메서드를 구현
 - 단, Use() 메서드의 내부 구현은 클래스마다 달라도 상관없음 - Use() 메서드의 구현은 아이템의 역할에 따라 달라짐

```
public class AmmoPack : MonoBehaviour, IItem {  
    public int ammo = 30;
```

```
    public void Use(GameObject target) {  
        // target에 탄알을 추가하는 처리  
        Debug.Log("탄알이 증가했다 : " + ammo);  
    }  
}
```

```
public class HealthPack : MonoBehaviour, IItem {  
    public float health = 50;
```

```
    public void Use(GameObject target) {  
        // target의 체력을 회복하는 처리  
        Debug.Log("체력을 회복했다 : " + health);  
    }  
}
```

15.1 인터페이스(3)

15.1.2 느슨한 커플링

- 느슨한 커플링(loose coupling) - 어떤 코드가 특정 클래스의 구현에 결합되지 않아 유연하게 변경 가능한 상태

```
void OnTriggerEnter(Collider other) {  
    AmmoPack ammoPack = other.GetComponent<AmmoPack>();  
    if (ammoPack != null) {  
        ammoPack.Use();  
    }  
  
    HealthPack healthPack = other.GetComponent<HealthPack>();  
    if (healthPack != null) {  
        healthPack.Use();  
    }  
}
```



```
void OnTriggerEnter(Collider other) {  
    IItem item = other.GetComponent<IItem>();  
    if (item != null) {  
        item.Use();  
    }  
}
```

- 인터페이스를 사용한 OnTriggerEnter() 메서드는 HealthPack과 AmmoPack의 구현에 더 이상 영향을 받지 않게 되었으므로 느슨한 커플링이 구현

15.1 인터페이스(4)

15.1.3 IDamageable

- 좀비 서바이버에서 공격당할 수 있는 모든 대상은 IDamageable 인터페이스를 상속해야 함
 - IDamageable 인터페이스는 프로젝트의 Scripts 폴더에 있음
- IDamageable 인터페이스를 상속하는 클래스는 OnDamage() 메서드를 반드시 구현
 - OnDamage() 메서드는 다음 입력을 받음
 - damage : 대미지 크기
 - hitPoint : 공격당한 위치
 - hitNormal : 공격당한 표면의 방향

```
using UnityEngine;
```

```
public interface IDamageable {  
    void OnDamage(float damage, Vector3 hitPoint, Vector3 hitNormal);  
}
```

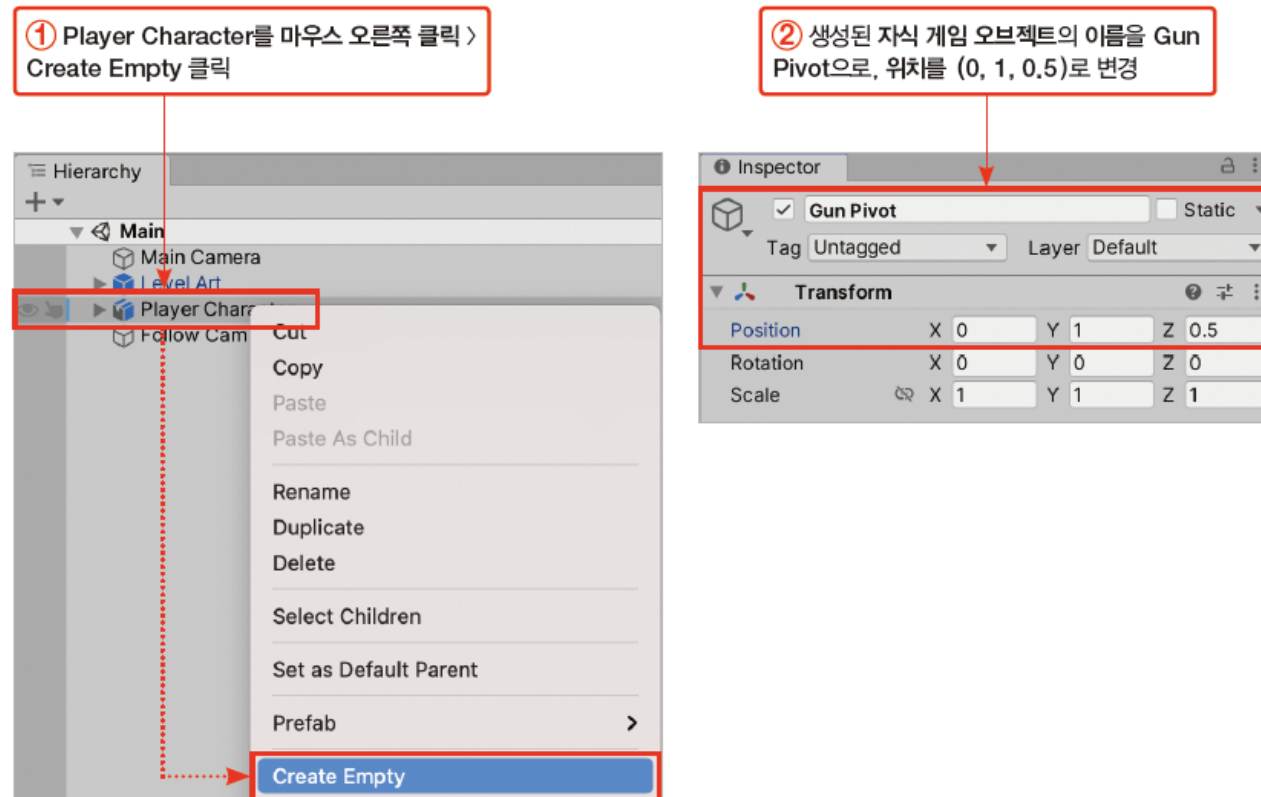
15.2 총 게임 오브젝트 준비(1)

15.2.1 Gun 게임 오브젝트 준비하기

- Player Character 게임 오브젝트에 총을 배치할 기준점이 될 자식 게임 오브젝트를 추가

[과정 01] Gun 배치 지점 생성

- ① 하이어라키 창에서 Player Character를 마우스 오른쪽 클릭 > Create Empty 클릭
- ② 생성된 자식 게임 오브젝트의 이름을 Gun Pivot으로, 위치를 (0, 1, 0.5)로 변경

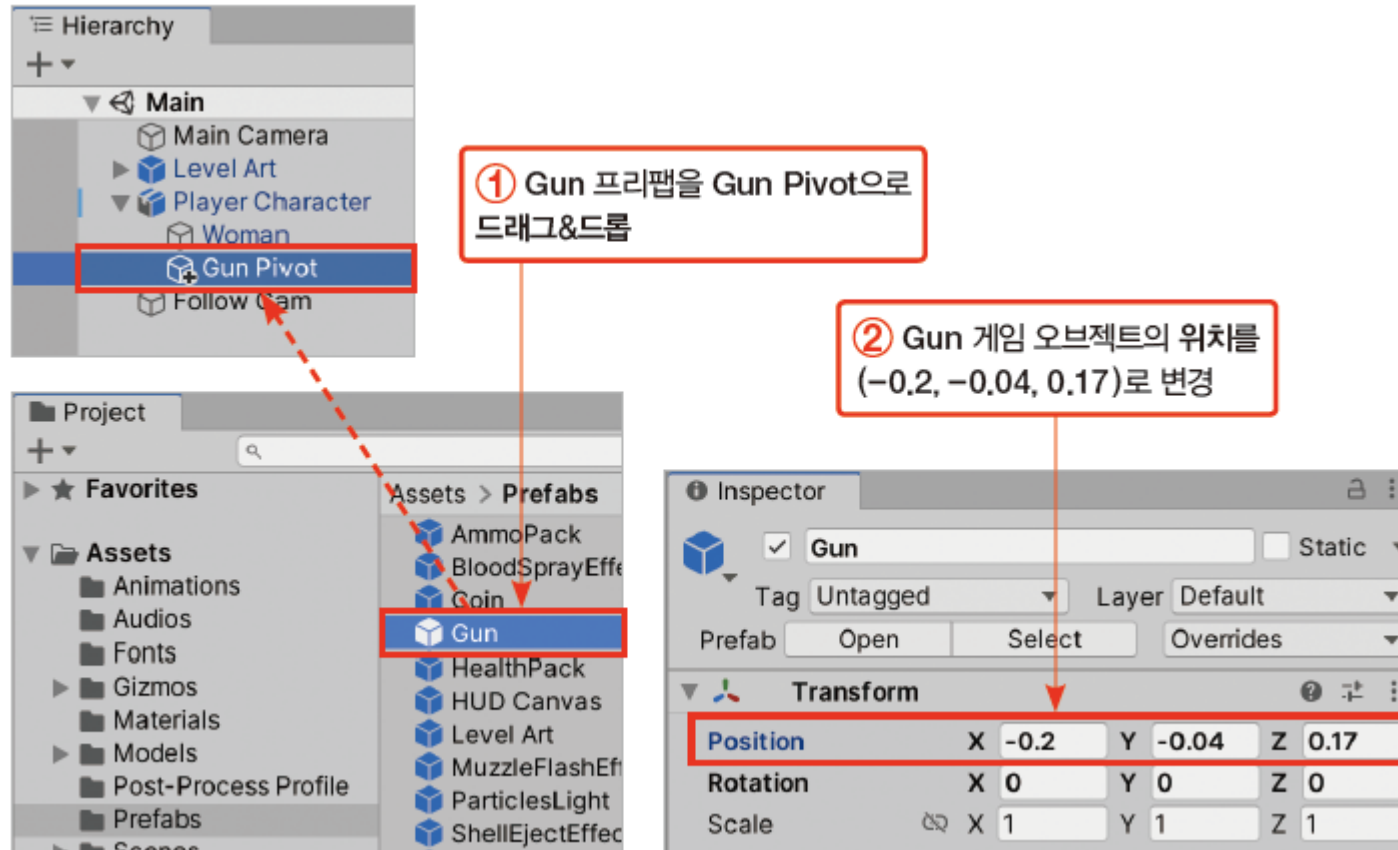


15.2 총 게임 오브젝트 준비(2)

- 총 프리팹을 이용해 총 게임 오브젝트를 추가

[과정 02] Gun 게임 오브젝트 생성

- ① Prefabs 폴더의 Gun 프리팹을 하이어라키 창의 Gun Pivot으로 드래그&드롭
- ② 생성된 Gun 게임 오브젝트의 위치를 (-0.2, -0.04, 0.17)로 변경



15.2 총 게임 오브젝트 준비(3)

- Gun Pivot 게임 오브젝트는 Gun 게임 오브젝트를 배치하는 데 사용할 기준점
 - 항상 오른쪽 팔꿈치에 위치
 - 현재는 Gun Pivot의 위치를 임의로 설정했지만, 15.5.7절 'OnAnimatorIK() 메서드'에서 Gun Pivot의 위치가 항상 캐릭터의 오른쪽 팔꿈치 위치로 이동하는 기능을 구현
- 앞에서 설정한 Gun 게임 오브젝트의 위치는 캐릭터가 견착 자세를 취하고 있는 상태에서 Gun Pivot이 오른쪽 팔꿈치에 위치할 경우 캐릭터 가슴 앞에 Gun 게임 오브젝트가 배치되도록 조정한 값
- 생성된 Gun 게임 오브젝트에는 여러 자식 게임 오브젝트가 있음• Model : 총 3D 모델
 - Left Handle : 캐릭터의 왼손이 위치할 곳
 - Right Handle : 캐릭터의 오른손이 위치할 곳
 - Fire Position : 탄알 발사 위치

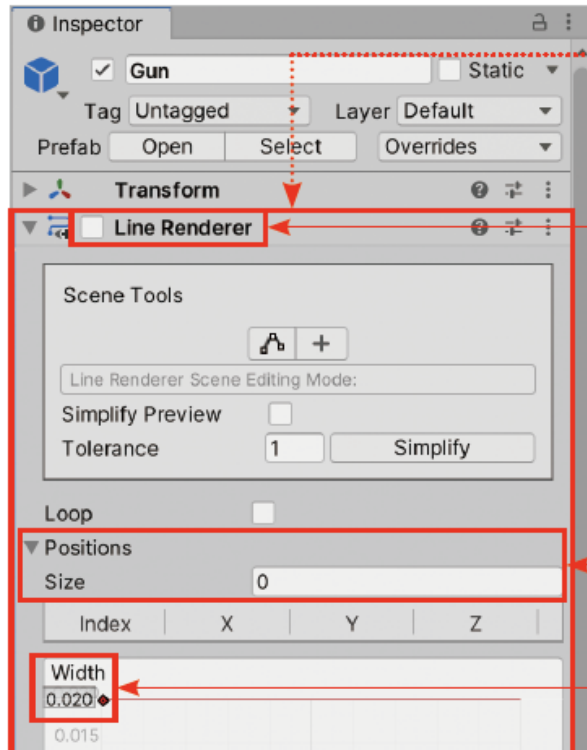
15.2 총 게임 오브젝트 준비(4)

- 총을 쏠 때 탄알의 궤적을 그리기 위한 라인 렌더러(Line Renderer) 추가

[과정 03] Gun에 라인 렌더러 추가하기

- ① Gun 게임 오브젝트에 Line Renderer 컴포넌트 추가(Add Component > Effects > Line Renderer)
- ② Line Renderer 컴포넌트를 체크 해제하여 비활성화
- ③ Positions 탭 펼치기 > Size를 0으로 변경
- ④ Width를 0.02로 변경
- ⑤ Materials 탭 펼치기 > Element 0에 Bullet 머티리얼 할당(Element 0 옆의 선택 버튼 클릭 > 선택 창에서 Bullet 머티리얼을 찾아 더블 클릭)
- ⑥ Lighting 탭의 Cast Shadows를 Off로 변경, Receive Shadows 체크 해제

15.2 총 게임 오브젝트 준비(5)

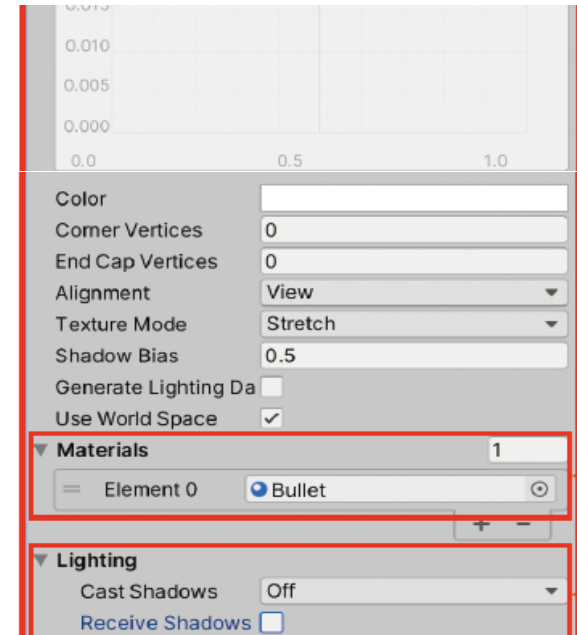


① Line Renderer 컴포넌트 추가
(Add Component > Effects > Line Renderer)

② Line Renderer 컴포넌트를 체크 해제하여 비활성화

③ Positions 탭 펼치기 > Size를 0으로 변경

④ Width를 0.02로 변경



⑤ Materials 탭 펼치기 > Element 0에 Bullet 머티리얼 할당

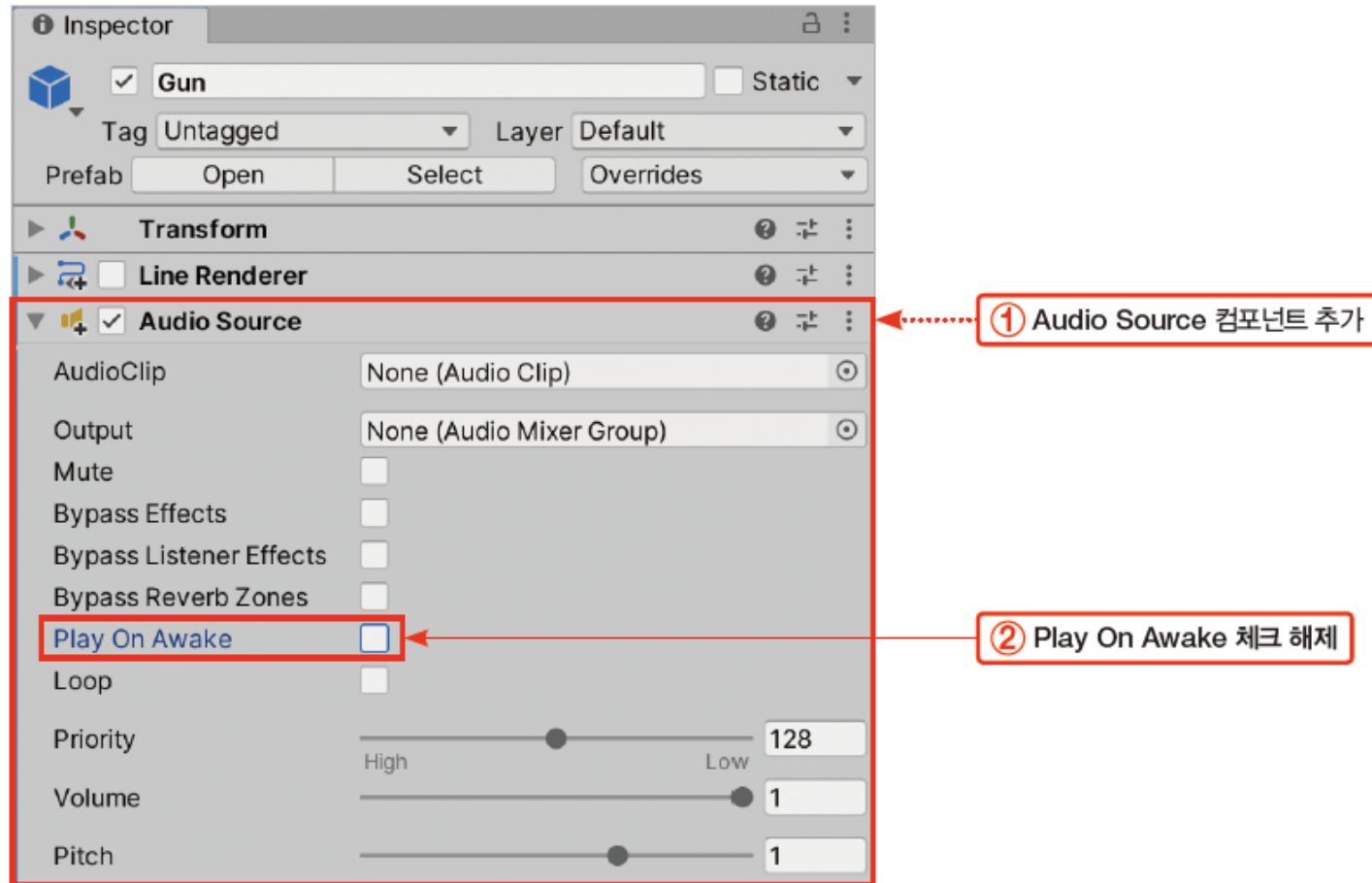
⑥ Cast Shadows를 Off로 변경, Receive Shadows 체크 해제

15.2 총 게임 오브젝트 준비(6)

- 총 쏘는 소리와 재장전 소리를 재생하는 오디오 소스 컴포넌트를 추가

[과정 04] Gun에 오디오 소스 추가하기

- ① Gun 게임 오브젝트에 Audio Source 컴포넌트 추가(Add Component > Audio > Audio Source)
- ② Audio Source 컴포넌트의 Play On Awake 체크 해제



15.2 총 게임 오브젝트 준비(7)

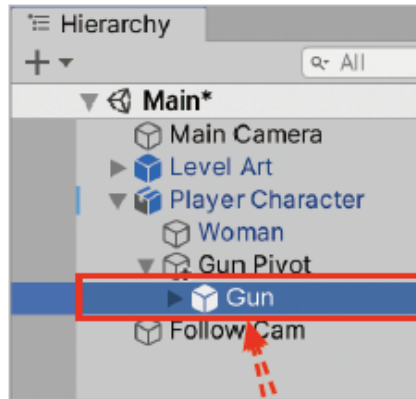
15.2.2 파티클 효과 추가하기

- 파티클 시스템(Particle System) 컴포넌트 - 유니티에서 연기, 화염재 등의 시각 효과
- 추가할 총구 화염 효과(MuzzleFlashEffect)와 탄피 배출 효과(ShellEjectEffect)는 미리 준비한 프리팹 사용

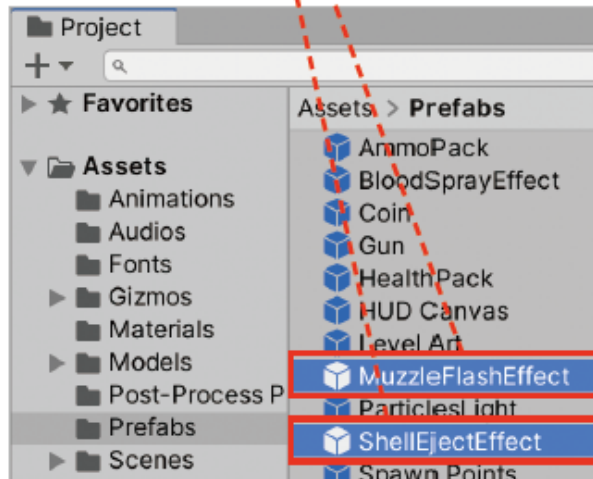
[과정 01] Gun에 사격 효과 추가하기

- ① Prefabs 폴더의 MuzzleFlashEffect와 ShellEjectEffect 프리팹을 하이어라키 창의 Gun 게임 오브젝트로 드래그&드롭
- ② MuzzleFlashEffect 게임 오브젝트의 위치를 (0, 0.08, 0.2)로 변경
- ③ ShellEjectEffect 게임 오브젝트의 위치를 (0.01, 0.09, -0.02)로 변경

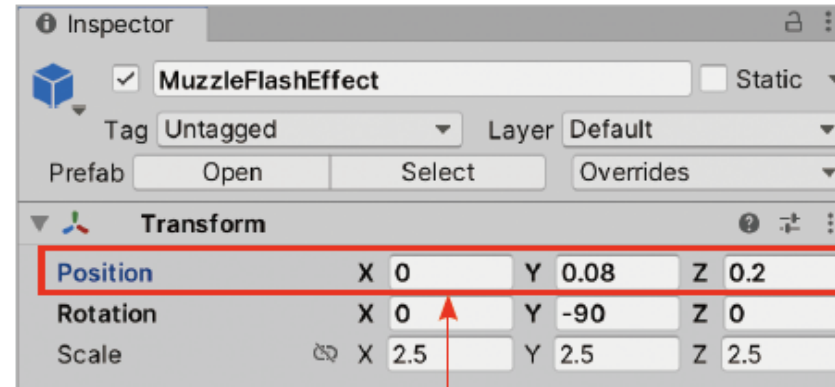
15.2 총 게임 오브젝트 준비(8)



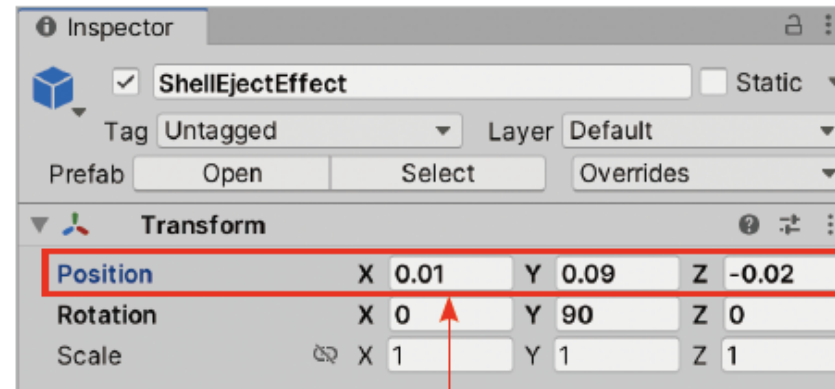
① MuzzleFlashEffect와 ShellEjectEffect 프리팹을 Gun 게임 오브젝트로 드래그&드롭



▶ Gun에 사격 효과 추가하기



② MuzzleFlashEffect 게임 오브젝트의 위치를 (0, 0.08, 0.2)로 변경



③ ShellEjectEffect 게임 오브젝트의 위치를 (0.01, 0.09, -0.02)로 변경

15.3 GunData 스크립트(1)

- 총의 동작을 구현하는 Gun 스크립트를 완성
- 여러 종류의 총을 만들 때를 대비하여 총에서 수치에 해당하는 데이터를 에셋 형태의 별개 오브젝트로 분리
- 만약 Gun 스크립트 내에 Gun의 모든 데이터가 변수로 선언되어 있으면 다음과 같은 문제가 발생
 - 총의 수치 데이터만 따로 관리하기 어려움
 - 같은 수치를 가진 같은 타입의 총들이 데이터를 각자 가지게 됨
 - 게임 도중에 총의 외형은 가만히 두고 데이터만 교체하기 힘들
- 이러한 문제를 해결하기 위해 스크립터블 오브젝트(Scriptable Object)를 사용하여 기존 Gun 스크립트를 개선

15.3 GunData 스크립트(2)

15.3.1 Gun의 확장성 문제

- 작성할 Gun 스크립트의 필수 부분이 다음과 같이 구성되어 있다고 가정

```
using System.Collections;
using UnityEngine;

public class Gun : MonoBehaviour {

    public AudioClip shotClip; // 발사 소리
    public AudioClip reloadClip; // 재장전 소리

    public float damage = 25; // 공격력
    public int magCapacity = 25; // 탄창 용량

    public float timeBetFire = 0.12f; // 탄알 발사 간격
    public float reloadTime = 1.8f; // 재장전 소요 시간

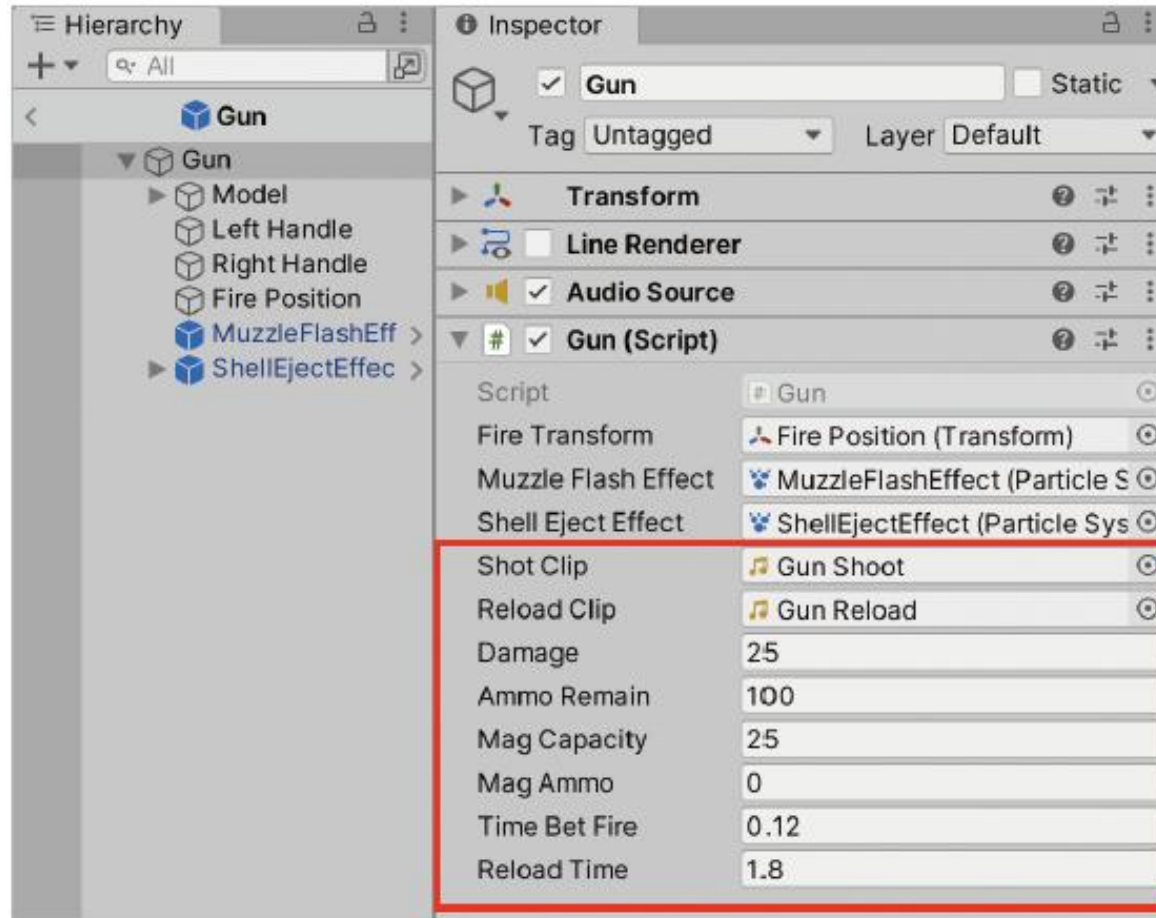
    // 이하 생략....
}
```

프로젝트를 확장하여 다양한 총을 구현하게 된다면 위 코드에서
다음 변수들은 총마다 다른 값을 가지게 됨

- 총의 사운드 : shotClip, reloadClip
- 공격력 : damage
- 탄창 용량 : magCapacity
- 연사력과 재장전 시간 관련 : timeBetFire, reloadTime

15.3 GunData 스크립트(3)

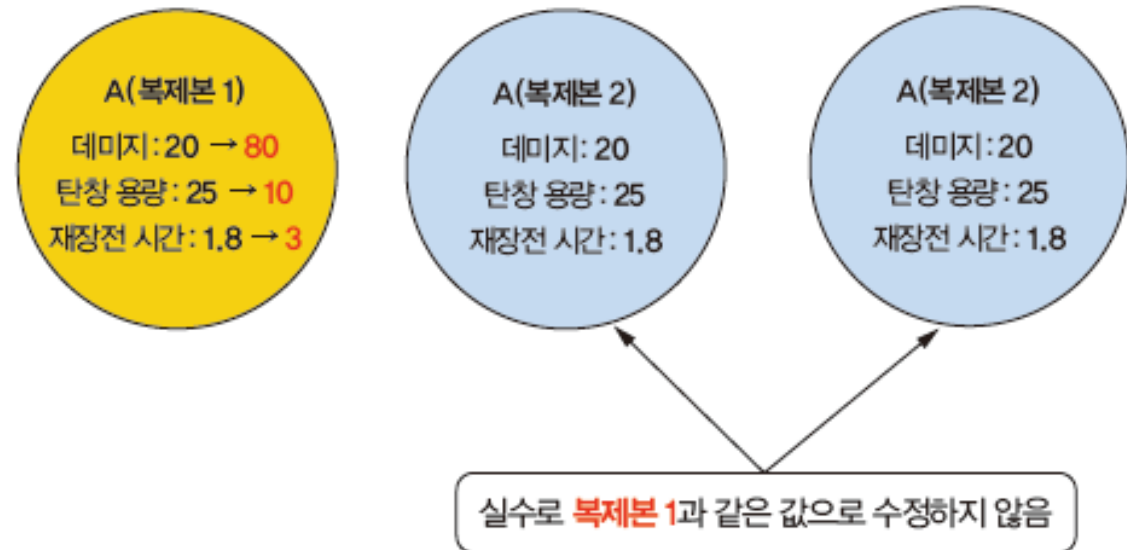
- 수치들은 Gun 컴포넌트에 종속되어 있으므로 완성한 Gun 게임 오브젝트나 프리팹을 찾아서 Gun 컴포넌트에서 아래 그림에 표시된 수치들을 직접 변경해야 함
 - 같은 타입의 총 게임 오브젝트가 여러 개 존재할 때, 각 총 게임 오브젝트마다 다른 수치를 할당하는 실수가 발생할 수 있음



15.3 GunData 스크립트(4)

- 씬에 A 총 프리팹으로부터 만들어진 여러 개의 A 총 게임 오브젝트가 있다고 가정
 - 어떤 A 총 게임 오브젝트의 수치를 수정하면 다른 A 총 게임 오브젝트의 수치도 함께 수정해야 함
 - 그렇지 않으면 다음과 같이 같은 종류의 총이 서로 다른 수치를 가질 수 있음

Inspector	Inspector
Gun A	Gun A (1)
Tag: Untagged	Tag: Untagged
Layer: Default	Layer: Default
Prefab: Open Select Overrides	Prefab: Open Select Overrides
Transform	Transform
Line Renderer	Line Renderer
Audio Source	Audio Source
Gun (Script)	Gun (Script)
Script: Gun	Script: Gun
Fire Transform: Fire Position	Fire Transform: Fire Position (T)
Muzzle Flash Effect: MuzzleFlash	Muzzle Flash Effect: MuzzleFlashEff
Shell Eject Effect: ShellEject	Shell Eject Effect: ShellEjectEffec
Shot Clip: Gun Shoot	Shot Clip: Gun Shoot
Reload Clip: Gun Reload	Reload Clip: Gun Reload
Damage: 80	Damage: 25
Ammo Remain: 100	Ammo Remain: 100
Mag Capacity: 10	Mag Capacity: 25
Mag Ammo: 0	Mag Ammo: 0
Time Bet Fire: 0.12	Time Bet Fire: 0.12
Reload Time: 3	Reload Time: 1.8



15.3 GunData 스크립트(5)

- 총의 데이터만 수정하려고 해도 총 게임 오브젝트나 프리팹을 수정해야 함
- 그리고 런타임에 총이 사용할 수치 데이터를 쉽게 교체할 수 없음
 - 예를 들어 하나의 총에 여러 발사 모드가 존재하는 경우
 - 다음과 같이 각 발사 모드 A와 B에 대응하는 데이터들을 필드로 선언하고 값을 할당하는 것은 불편

```
using System.Collections;
using UnityEngine;

public class Gun : MonoBehaviour {

    // MODE A
    public AudioClip shotClipA; // 모드 A 발사 소리
    public AudioClip reloadClipA; // 모드 A 재장전 소리
```

```
public float damageA = 25; // 모드 A 공격력
private float fireDistanceA = 50f; // 모드 A 사정거리

// MODE B
public AudioClip shotClipB; // 모드 B 발사 소리
public AudioClip reloadClipB; // 모드 B 재장전 소리

public float damageB = 25; // 모드 B 공격력
private float fireDistanceB = 50f; // 모드 B 사정거리

// 생략...
}
```

15.3 GunData 스크립트(6)

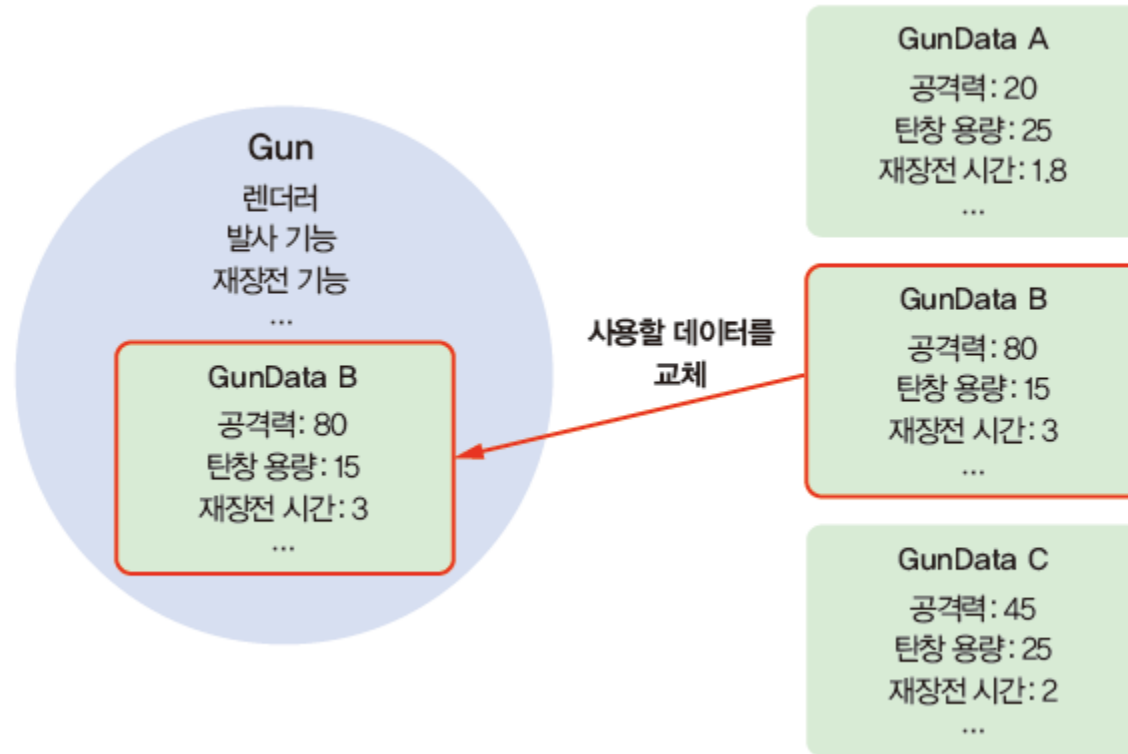
- 총의 외형은 같은데 총의 수치 데이터만 다른 경우
 - 예를 들어 기존 총에서 성능이 강화된 총이 존재
 - 같은 총에 대한 프리팹 두 개가 필요
 - 외형은 같지만 Gun 컴포넌트의 필드에 할당된 수치 값만 다른 프리팹을 두 개 만들어야 함

Inspector	Inspector
☒ Gun A	☒ Gun A -Upgrade
Tag Untagged Layer Default	Tag Untagged Layer Default
Prefab Open Select Overrides	Prefab Open Select Overrides
Transform	Transform
Line Renderer	Line Renderer
Audio Source	Audio Source
Gun (Script)	Gun (Script)
Script # Gun	Script # Gun
Fire Transform Fire Position (Transform)	Fire Transform Fire Position (Transform)
Muzzle Flash Effect MuzzleFlashEffect (ParticleSystem)	Muzzle Flash Effect MuzzleFlashEffect (ParticleSystem)
Shell Eject Effect ShellEjectEffect (ParticleSystem)	Shell Eject Effect ShellEjectEffect (ParticleSystem)
Shot Clip Gun Shoot	Shot Clip Gun Shoot
Reload Clip Gun Reload	Reload Clip Gun Reload
Damage 25	Damage 50
Ammo Remain 100	Ammo Remain 120
Mag Capacity 25	Mag Capacity 50
Mag Ammo 0	Mag Ammo 0
Time Bet Fire 0.12	Time Bet Fire 0.1
Reload Time 3	Reload Time 1.8

15.3 GunData 스크립트(7)

15.3.2 Gun과 GunData 분리

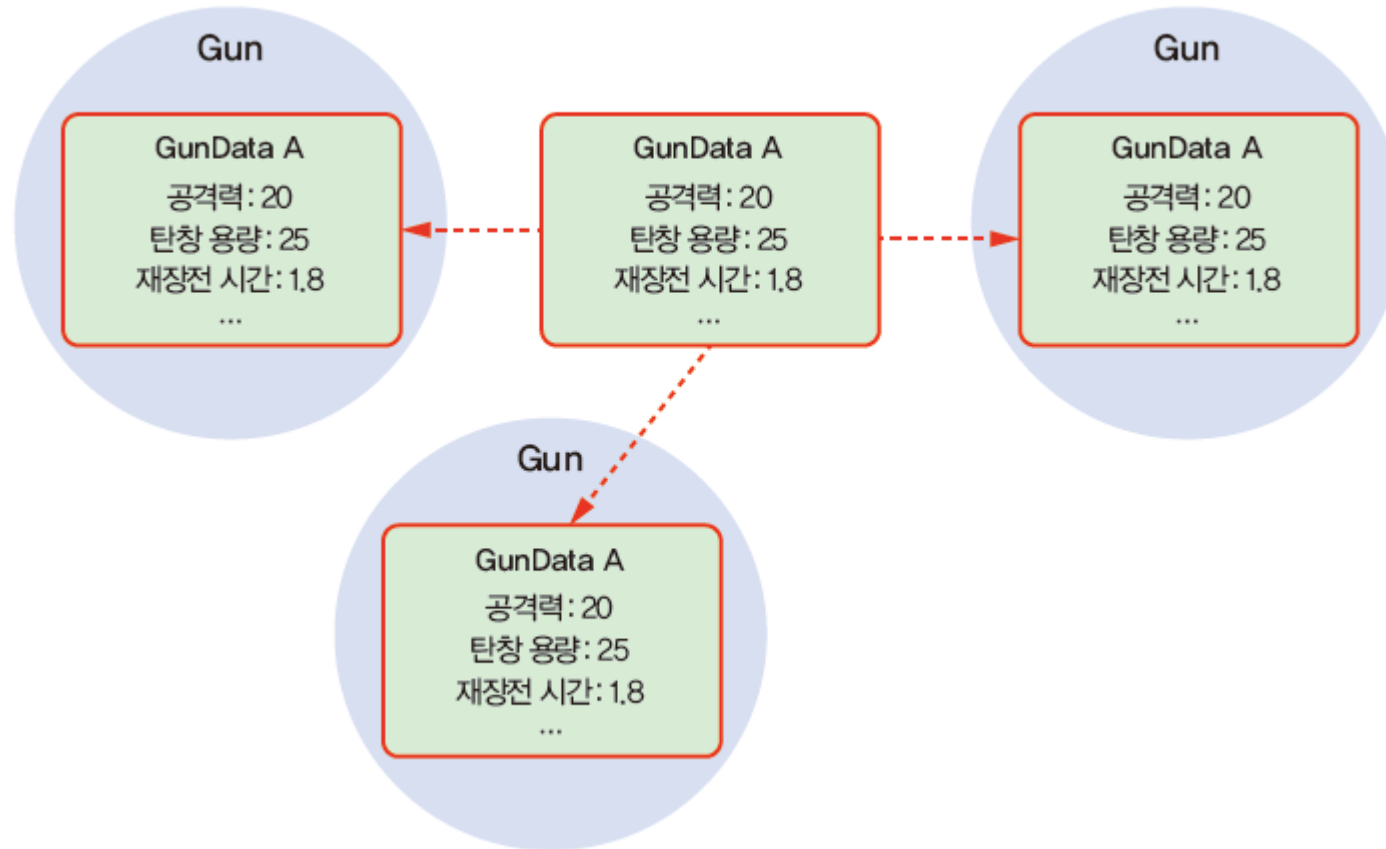
- Gun 클래스에서 데이터에 해당하는 부분을 개별 클래스 GunData로 추출하여 일부 해결
- Gun 게임 오브젝트를 수정하지 않고 Gun 게임 오브젝트가 사용할 GunData만 교체



▶ Gun과 GunData 분리(1)

15.3 GunData 스크립트(8)

- 다수의 Gun 게임 오브젝트가 하나의 GunData를 참조



▶ Gun과 GunData 분리(2)

15.3 GunData 스크립트(9)

- 문제를 완전히 해결하려면
 - GunData 오브젝트가 유니티 에디터에서 편집할 수 있는 형태로 존재
 - 씬 위의 게임 오브젝트가 아닌 형태로 존재
- GunData는 간단한 데이터 컨테이너
 - 따라서 MonoBehaviour를 상속받아서 안 됨
 - 스크립터블 오브젝트로 해결
 - 여러 오브젝트가 공유하여 사용할 데이터를 에셋 형태로 분리
 - 데이터를 유니티 인스펙터 창에서 편집 가능한 형태로 관리

15.3 GunData 스크립트(10)

The image shows a Unity Hierarchy panel on the left and two Inspector panels on the right. The Hierarchy panel shows a tree structure with 'Assets > ScriptableData' containing 'Gun Heavy', 'Gun Rifle', and 'Gun SMG'. Red boxes highlight 'Gun Heavy' and 'Gun Rifle'. Red dotted arrows point from these boxes to the corresponding Inspector panels. The top Inspector panel is for 'Gun Heavy (Gun Data)' and the bottom is for 'Gun Rifle (Gun Data)'. Both show the 'GunData' script with various parameters. A red box highlights the 'Shot Clip' and 'Reload Clip' fields in the Gun Rifle Inspector, with a red arrow pointing to a text box below.

오디오 클립 등 유니티 에셋 & 오브젝트도 할당 가능

▶ GunData 타입의 스크립터블 오브젝트 에셋들

Property	Gun Heavy (Gun Data)	Gun Rifle (Gun Data)
Script	GunData	GunData
Shot Clip	Gun Shoot	Gun Shoot
Reload Clip	Gun Reload	Gun Reload
Damage	10	25
Start Ammo Remain	250	100
Mag Capacity	10	25
Time Bet Fire	0.05	0.12
Reload Time	5	1.8

15.3 GunData 스크립트(11)

15.3.4 GunData 구현하기

[과정 01] GunData 스크립트 열기

① Scripts 폴더의 GunData 스크립트를 더블 클릭으로 열기

```
using UnityEngine;

public class GunData
{
    public AudioClip shotClip; // 발사 소리
    public AudioClip reloadClip; // 재장전 소리

    public float damage = 25; // 공격력

    public int startAmmoRemain = 100; // 처음에 주어질 전체 탄알
    public int magCapacity = 25; // 탄창 용량

    public float timeBetFire = 0.12f; // 탄알 발사 간격
    public float reloadTime = 1.8f; // 재장전 소요 시간
}
```

15.3 GunData 스크립트(12)

- 앞의 GunData 스크립트 설명

- shotClip과 reloadClip에는 총 쏘는 소리 오디오 클립과 재장전 소리 오디오 클립이 각각 할당
- 그다음 총의 수치를 저장하는 변수가 선언

```
public float damage = 25; // 공격력
```

```
public int startAmmoRemain = 100; // 처음에 주어질 전체 탄알
```

```
public int magCapacity = 25; // 탄창 용량
```

- 시간과 관련된 변수가 선언

```
public float timeBetFire = 0.12f; // 탄알 발사 간격
```

```
public float reloadTime = 1.8f; // 재장전 소요 시간
```

15.3 GunData 스크립트(13)

ScriptableObject로 만들기

- GunData를 스크립터블 오브젝트 에셋으로 생성할 수 있도록 스크립터블 오브젝트 타입으로 만들기

[과정 01] GunData 스크립트를 ScriptableObject로 만들기

① GunData 스크립트 상단을 다음과 같이 수정

- GunData가 스크립터블 오브젝트로 동작할 수 있도록 GunData 클래스가 ScriptableObject 클래스를 상속

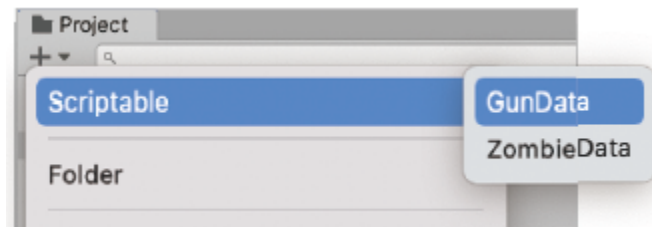
```
public class GunData : ScriptableObject
```

- 유니티의 에셋 생성 메뉴에서 GunData 타입의 에셋을 생성하는 메뉴를 만들기 위해 다음과 같은 특성(Attribute)을 클래스에 추가

```
[CreateAssetMenu(menuName = "Scriptable/GunData", fileName = "Gun Data")]
```

- CreateAssetMenu 특성은 해당 타입의 에셋을 생성할 수 있는 버튼을 Assets와 Create (+ 버튼) 메뉴에 추가

```
[CreateAssetMenu(menuName = "메뉴 경로", fileName = "기본 파일명", order = 메뉴 상에서 순서)]
```



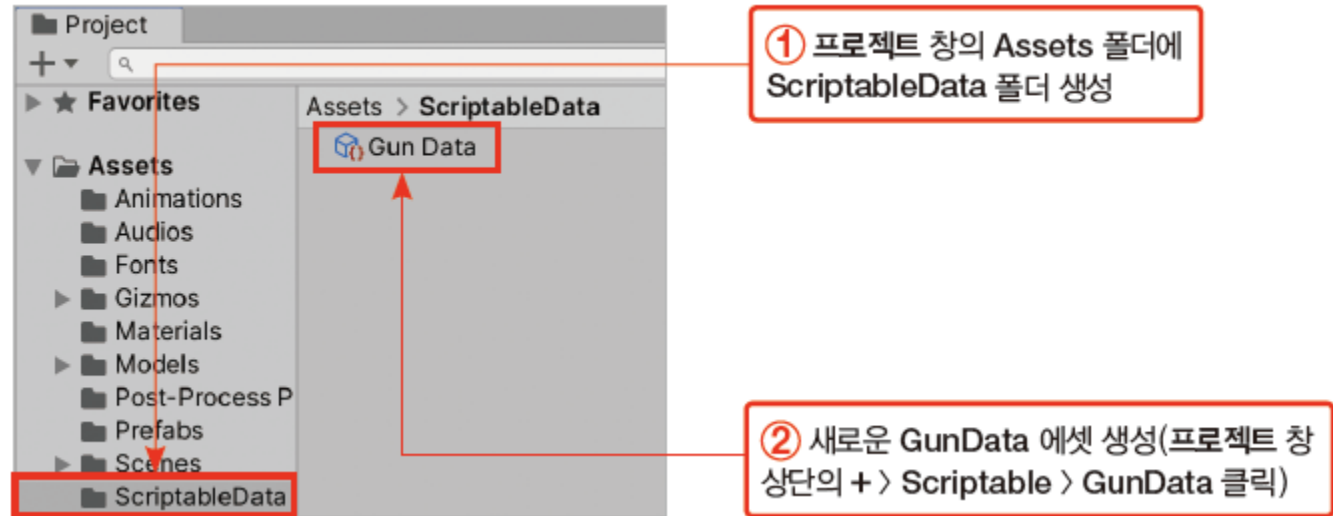
15.3 GunData 스크립트(14)

15.3.5 GunData 에셋 생성하기

- GunData 스크립터블 오브젝트로부터 Gun에 사용할 데이터 에셋을 미리 생성

[과정 01] GunData 에셋 생성하기

- ① 프로젝트 창의 Assets 폴더에 ScriptableData 폴더 생성 → ScriptableData 폴더로 이동
- ② 새로운 GunData 에셋 생성(프로젝트 창 상단의 + > Scriptable > GunData 클릭)

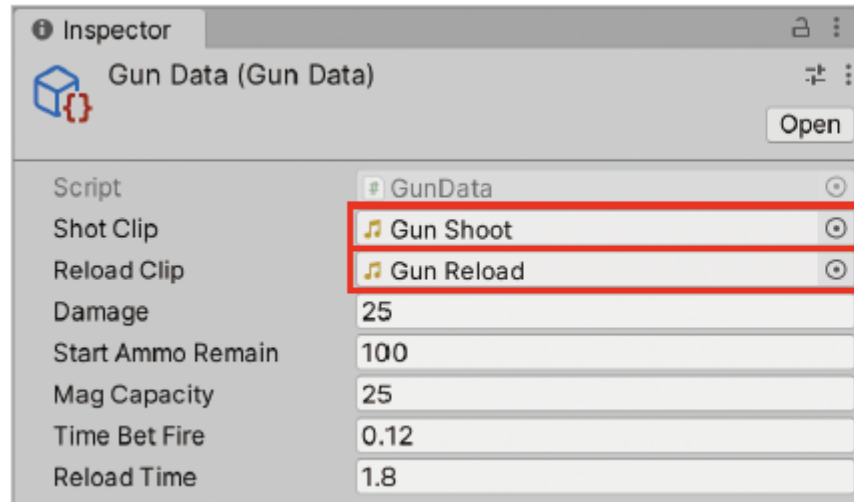


15.3 GunData 스크립트(15)

- Gun Data 에셋의 필드를 완성

[과정 02] Gun Data의 필드 완성하기

- ① Shot Clip 필드에 Gun Shoot 오디오 클립 할당(필드 옆 선택 버튼 사용)
- ② Reload Clip 필드에 Gun Reload 오디오 클립 할당(필드 옆 선택 버튼 사용)



② Shot Clip 필드에 Gun Shoot 오디오 클립 할당(필드 옆 선택 버튼 사용)

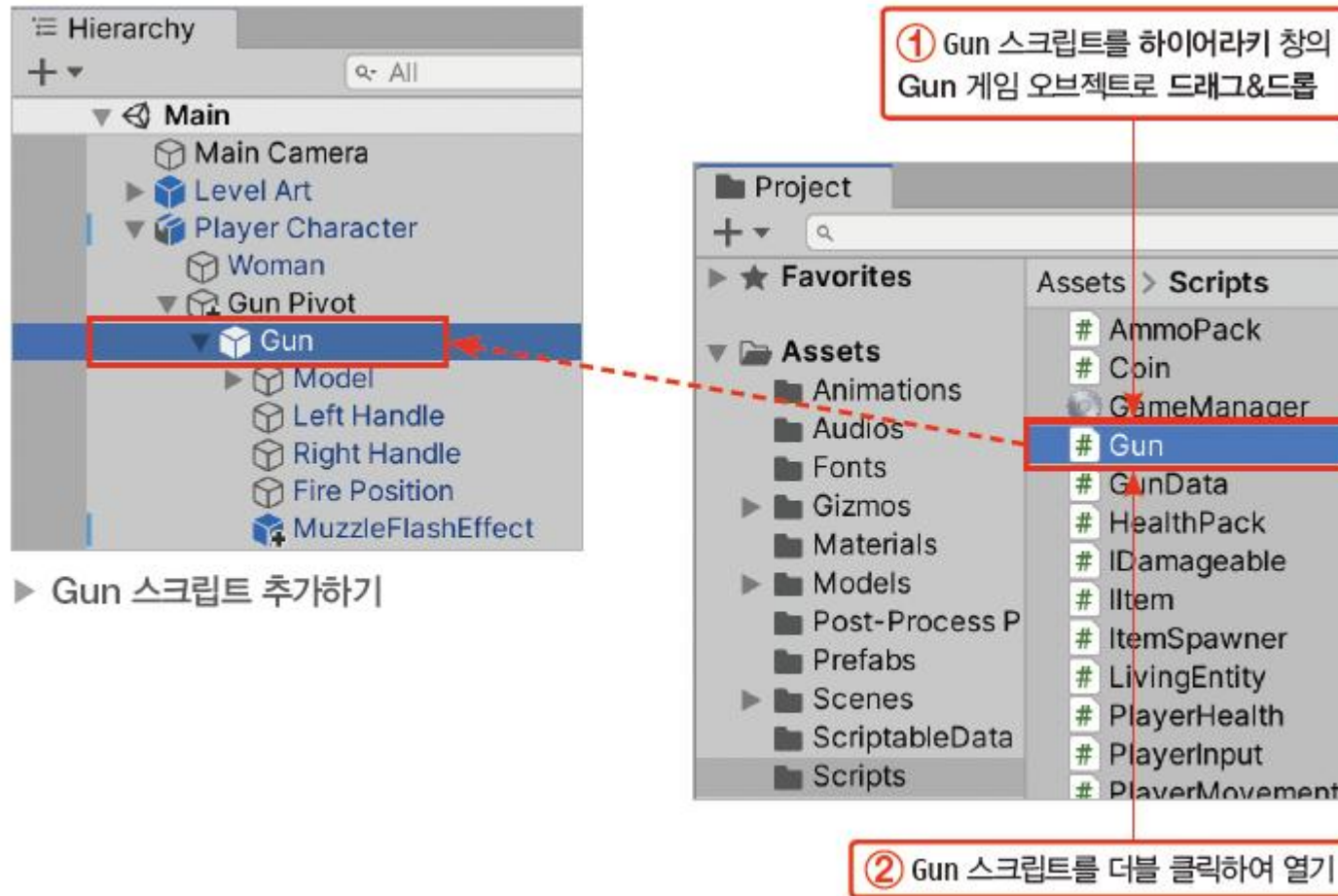
① Reload Clip 필드에 Gun Reload 오디오 클립 할당(필드 옆 선택 버튼 사용)

15.4 Gun 스크립트(1)

- 총의 동작을 구현하는 Gun 스크립트를 Gun 게임 오브젝트에 추가하고 스크립트를 완성

[과정 01] Gun 스크립트 추가하기

- ① Scripts 폴더의 Gun 스크립트를 하이어라키 창의 Gun 게임 오브젝트로 드래그&드롭
- ② Gun 스크립트를 더블 클릭하여 열기



(전체 스크립트 생략)

15.4 Gun 스크립트(2)

15.4.1 Gun의 메서드

- Gun 스크립트의 메서드 이름과 역할
 - Awake() 메서드 - 사용할 컴포넌트를 가져오고 OnEnable() 메서드는 총의 상태를 초기화
 - Fire() 메서드 - 총을 발사
 - 실제 발사 처리는 Shot() 메서드에서 이루어지며 Fire() 메서드는 Shot() 메서드를 안전하게 감싸는 껍데기
 - Gun 클래스 외부에서는 public인 Fire() 메서드를 사용해 발사를 시도
 - Fire() 메서드는 총이 발사 가능한 상태에서만 Shot() 메서드를 실행
 - ShotEffect() 메서드 - 발사 효과를 재생하고 탄알 궤적을 그림
 - Reload() 메서드 - 재장전을 시도하는 메서드
 - 실제 재장전은 ReloadRoutine() 메서드에서 이루어지며, Reload() 메서드는 재장전이 가능할 때만 실제 재장전이 이루어지도록 ReloadRoutine() 메서드를 감싸는 역할
 - ShotEffect() 메서드와 ReloadRoutine() 메서드 - IEnumerator라는 코루틴 메서드를 반환

15.4 Gun 스크립트(3)

15.4.2 Gun의 필드

- 총의 현재 상태를 표현하는 State라는 새로운 enum 타입을 정의

```
public enum State {  
    Ready, // 발사 준비됨  
    Empty, // 탄창이 빈  
    Reloading // 재장전 중  
}
```

- 위에서 정의한 State 타입으로 gunState라는 변수를 선언했다고 가정
 - gunState에는 다음과 같이 총의 현재 상태에 따라 서로 다른 값을 할당

```
State gunState;  
gunState = State.Ready; // 현재 총이 발사 준비된 상태  
gunState = State.Empty; // 현재 총의 탄창이 빈 상태  
gunState = State.Reloading; // 현재 총이 재장전 중인 상태
```

- 총의 상태를 표현하는 state 프로퍼티를 선언

```
public State state { get; private set; } // 현재 총의 상태
```

15.4 Gun 스크립트(4)

- state 프로퍼티 아래에는 Gun 게임 오브젝트로부터 가져와서 사용할 컴포넌트들에 대한 변수가 선언

```
public Transform fireTransform; // 탄알이 발사될 위치
```

```
public ParticleSystem muzzleFlashEffect; // 총구 화염 효과
```

```
public ParticleSystem shellEjectEffect; // 탄피 배출 효과
```

```
private LineRenderer bulletLineRenderer; // 탄알 궤적을 그리기 위한 렌더러
```

```
private AudioSource gunAudioPlayer; // 총 소리 재생기
```

- 총의 데이터를 저장하는 GunData 타입의 변수가 선언

```
public GunData gunData; // 총의 현재 데이터
```

- gunData.shotClip : 발사 소리
- gunData.reloadClip : 재장전 소리
- gunData.damage : 공격력
- gunData.startAmmoRemain : 시작시 주어질 탄알

- gunData.magCapacity : 탄창 용량
- gunData.timeBetFire : 탄알 발사 간격
- gunData.reloadTime : 재장전 소요 시간

15.4 Gun 스크립트(5)

- 현재 총의 상태를 나타내는 변수들이 선언

```
private float fireDistance = 50f; // 사정거리
```

```
public int ammoRemain = 100; // 남은 전체 탄알
```

```
public int magAmmo; // 현재 탄창에 남아 있는 탄알
```

- lastFireTime은 총을 마지막으로 발사한 시점이며, 연사를 구현할 때 사용

```
private float lastFireTime; // 총을 마지막으로 발사한 시점
```

15.4 Gun 스크립트(6)

15.4.3 Awake() 메서드

- Gun 스크립트의 Awake() 메서드를 완성

[과정 01] Awake() 메서드 완성하기

- ① Gun 스크립트의 Awake() 메서드를 다음과 같이 완성

```
private void Awake() {  
    // 사용할 컴포넌트의 참조 가져오기  
    gunAudioPlayer = GetComponent();  
    bulletLineRenderer = GetComponent<LineRenderer>();  
  
    // 사용할 점을 두 개로 변경  
    bulletLineRenderer.positionCount = 2;  
    // 라인 렌더러를 비활성화  
    bulletLineRenderer.enabled = false;  
}
```

◀ GetComponent() 메서드로 오디오 소스 컴포넌트와 라인 렌더러 컴포넌트를 Gun 게임 오브젝트로부터 가져오기

◀ 라인 렌더러가 사용할 점의 수를 2로 변경하고, 라인 렌더러 컴포넌트를 미리 비활성화

15.4 Gun 스크립트(7)

15.4.4 OnEnable() 메서드

- OnEnable() 메서드는 컴포넌트가 활성화될 때마다 실행

[과정 01] OnEnable () 메서드 완성하기

- ① Gun 스크립트의 OnEnable() 메서드를 다음과 같이 완성

```
private void OnEnable() {  
    // 전체 예비 탄알 양을 초기화  
    ammoRemain = gunData.startAmmoRemain;  
    // 현재 탄창을 가득 채우기  
    magAmmo = gunData.magCapacity;  
  
    // 총의 현재 상태를 총을 쏠 준비가 된 상태로 변경  
    state = State.Ready;  
    // 마지막으로 총을 쏜 시점을 초기화  
    lastFireTime = 0;  
}
```

- ◀ 총이 활성화될 때 먼저 전체 예비 탄창을 초기화
- ◀ 현재 탄창에 탄알을 최대 용량까지 채움

- ◀ 총의 현재 상태를 발사 준비된 상태로 만들고, 마지막 발사 시점을 초기화

15.4 Gun 스크립트(8)

15.4.5 코루틴

- 유니티의 코루틴(Coroutine) 메서드는 대기 시간을 가질 수 있는 메서드
 - IEnumerator 타입을 반환해야 하며, 처리가 일시 대기할 곳에 yield 키워드를 명시

코루틴의 동작 원리

- CleaningHouse() 메서드를 실행하면 처음부터 마지막 줄까지 중간에 멈추지 않고 처리가 진행
 - 따라서 A, B, C가 한 번에 모두 청소
- 코루틴은 집의 구역을 나누어 오늘은 어떤 구역을 청소하고 쉰 다음 내일은 남은 구역을 청소하는 방식

```
void CleaningHouse() {  
    // A방 청소  
    // B방 청소  
    // C방 청소  
}
```



```
IEnumerator CleaningHouse() {  
    // A방 청소  
    yield return new WaitForSeconds(10f); // 10초 동안 쉬기  
    // B방 청소  
    yield return new WaitForSeconds(20f); // 20초 동안 쉬기  
    // C방 청소  
}
```

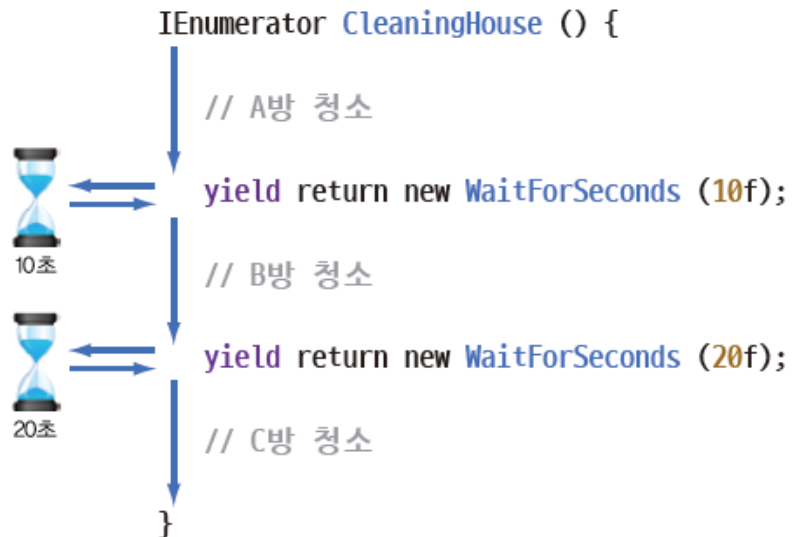
15.4 Gun 스크립트(9)

- 코루틴 메서드는 StartCoroutine() 메서드로 실행

```
StartCoroutine(CleaningHouse());
```

- CleaningHouse() 코루틴을 실행하면 먼저 A를 청소
 - 그리고 다음의 yield 문을 만나는 순간 10초 동안 코루틴 진행이 일시 정지
 - 대기 시간이 끝나면 프로그램의 처리가 코루틴의 마지막 실행 줄로 되돌아가고 남은 처리를 이어서 진행

```
yield return new WaitForSeconds(10f);
```



- A방 청소 → 10초 대기 → B방 청소 → 20초 대기 → C방 청소

15.4 Gun 스크립트(10)

15.4.6 ShotEffect() 메서드

- 코루틴을 사용해 ShotEffect() 메서드를 완성

[과정 01] ShotEffect () 메서드 완성하기

① Gun 스크립트의 ShotEffect() 메서드를 다음과 같이 완성

```
private IEnumerator ShotEffect(Vector3 hitPosition) {  
    // 총구 화염 효과 재생  
    muzzleFlashEffect.Play();  
    // 탄피 배출 효과 재생  
    shellEjectEffect.Play();  
  
    // 총격 소리 재생  
    gunAudioPlayer.PlayOneShot(gunData.shotClip);
```

```
    // 선의 시작점은 총구의 위치  
    bulletLineRenderer.SetPosition(0, fireTransform.position);  
    // 선의 끝점은 입력으로 들어온 총돌 위치  
    bulletLineRenderer.SetPosition(1, hitPosition);  
    // 라인 렌더러를 활성화하여 탄알 궤적을 그림  
    bulletLineRenderer.enabled = true;  
  
    // 0.03초 동안 잠시 처리를 대기  
    yield return new WaitForSeconds(0.03f);  
  
    // 라인 렌더러를 비활성화하여 탄알 궤적을 지움  
    bulletLineRenderer.enabled = false;  
}
```


15.4 Gun 스크립트(11)

- 완성된 ShotEffect() 코루틴 메서드

- 총구 화염 효과와 탄피 배출 효과를 재생
 - 파티클 시스템 컴포넌트의 효과는 Play() 메서드로 재생

```
muzzleFlashEffect.Play();  
shellEjectEffect.Play();
```

- 총 쏘는 소리 오디오 클립을 gunData.shotClip으로 가져와 오디오 소스의 PlayOneShot() 메서드로 재생

```
gunAudioPlayer.PlayOneShot(gunData.shotClip);
```

- 탄알 궤적 그리기

```
bulletLineRenderer.SetPosition(0, fireTransform.position);  
bulletLineRenderer.SetPosition(1, hitPosition);  
bulletLineRenderer.enabled = true;
```

- yield 문을 사용해 0.03초 동안 처리를 일시 정지 - 그동안 라인 렌더러는 켜진 채로 유지

```
yield return new WaitForSeconds(0.03f);  
bulletLineRenderer.enabled = false;
```

15.4 Gun 스크립트(12)

15.4.7 Fire() 메서드

- 총 발사를 시도하는 Fire() 메서드를 완성
 - Fire() 메서드는 총을 발사 가능한 상태에서만 Shot() 메서드가 실행되도록 감싸는 역할

[과정 01] Fire () 메서드 완성하기

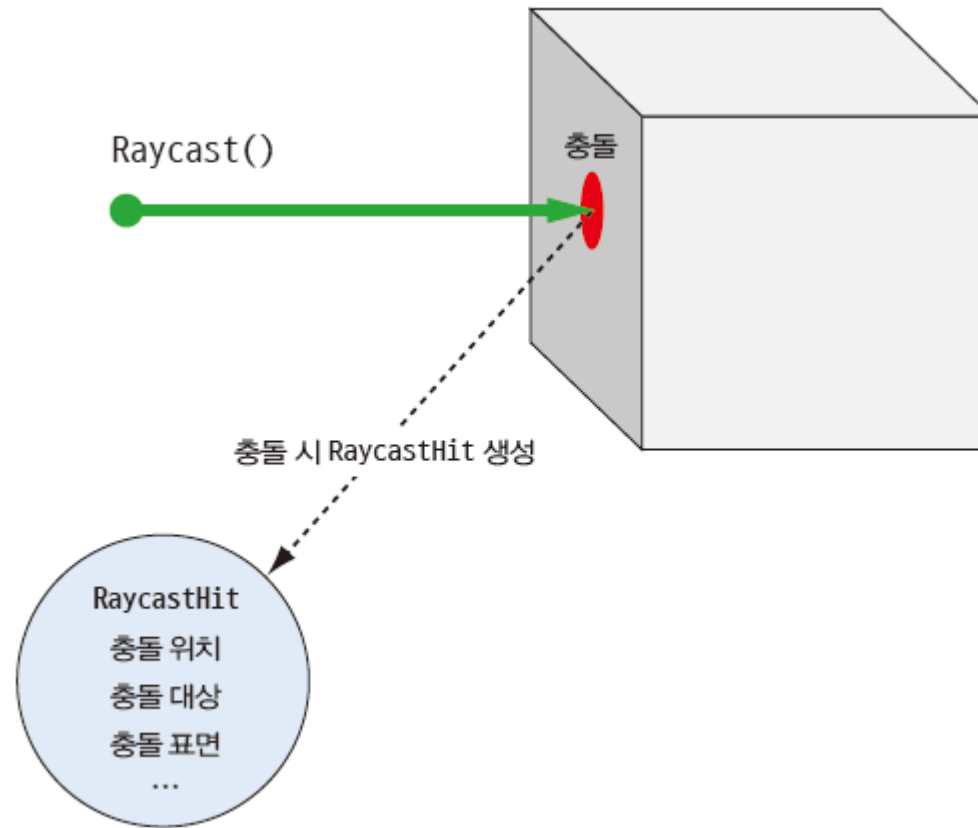
- ① Gun 스크립트의 Fire() 메서드를 다음과 같이 완성

```
public void Fire() {  
    // 현재 상태가 발사 가능한 상태  
    // && 마지막 총 발사 시점에서 gunData.timeBetFire 이상의 시간이 지남  
    if (state == State.Ready && Time.time >= lastFireTime + gunData.timeBetFire)  
    {  
        // 마지막 총 발사 시점 갱신  
        lastFireTime = Time.time;  
        // 실제 발사 처리 실행  
        Shot();  
    }  
}
```

15.4 Gun 스크립트(13)

15.4.8 레이캐스트

- 레이캐스트(Raycast)는 보이지 않는 광선을 쏘을 때 광선이 다른 콜라이더와 충돌하는지 검사하는 처리
- 이때 사용하는 광선을 레이라고 부르며 Ray 타입으로 레이의 정보만 따로 표현할 수 있음



15.4 Gun 스크립트(14)

15.4.9 Shot() 메서드

- 레이캐스트를 이용해 총을 쏘고, 총에 맞은 오브젝트를 찾아 대미지를 주는 Shot() 메서드를 완성

[과정 01] Shot () 메서드 완성하기

① Gun 스크립트의 Shot() 메서드를 다음과 같이 완성 (전체 코드 생략)

- 레이캐스트의 결과를 저장할 변수를 선언

```
RaycastHit hit;
```

```
Vector3 hitPosition = Vector3.zero;
```

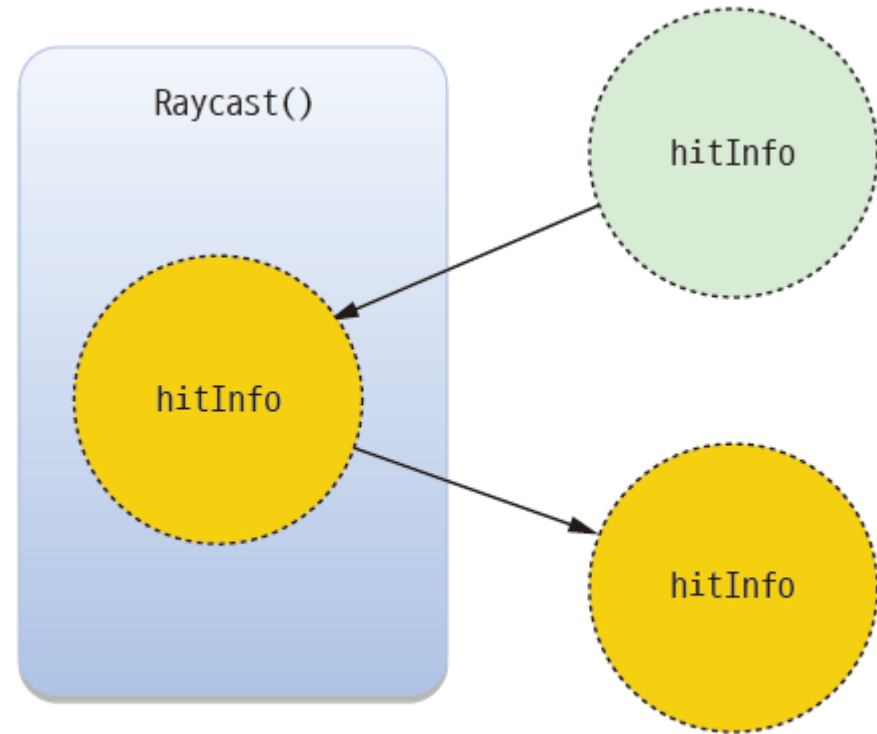
- Raycast() 메서드는 입력을 다양한 형태로 받을 수 있음

```
Raycast(Vector3 origin, Vector3 direction, out RaycastHit hitInfo, float maxDistance);
```

- Vector3 origin : 레이의 시작점
- Vector3 direction : 레이의 방향
- RaycastHit hitInfo : 레이가 충돌한 경우 hitInfo에 자세한 충돌 정보가 채워짐
- float maxDistance : 레이 충돌을 검사할 최대 거리

15.4 Gun 스크립트(15)

- Raycast() 메서드는 자신의 내부에서 hitInfo에 충돌 정보를 채움
 - Raycast() 메서드가 종료되었을 때 변경 사항이 유지된 채로 hitInfo가 되 돌아옴
- out 키워드는 메서드가 return 이외의 방법으로 추가 정보를 반환할 수 있게 해줌
- Raycast() 메서드에 다음 값을 입력해서 실행
 - origin : fireTransform.position(총구 위치)
 - direction : fireTransfom.forward(총구의 앞쪽 방향)
 - hitInfo : hit(충돌 정보 컨테이너)
 - maxDistance : fireDistance(사정거리)



▶ out 키워드의 동작

15.4 Gun 스크립트(16)

- 레이캐스트를 실행하고 충돌 여부를 if 문으로 검사

```
if (Physics.Raycast(fireTransform.position, fireTransform.forward, out hit, fireDistance))
{
    IDamageable target =
        hit.collider.GetComponent<IDamageable>();

    if (target != null)
    {
        target.OnDamage(gunData.damage, hit.point, hit.normal);
    }

    hitPosition = hit.point;
}
else
{
    hitPosition = fireTransform.position + fireTransform.forward * fireDistance;
}
```

15.4 Gun 스크립트(17)

- 레이가 어떤 게임 오브젝트의 콜라이더와 충돌하면 if 문 블록이 실행되고, 충돌한 상대방 게임 오브젝트로부터 IDamageable 타입의 컴포넌트를 가져와 target에 할당을 시도

```
IDamageable target = hit.collider.GetComponent<IDamageable>();
```

```
if (target != null)
{
    target.OnDamage(gunData.damage, hit.point, hit.normal);
}
```

- 상대방 컴포넌트의 OnDamage() 메서드를 실행하고 대미지, 탄알이 맞은 위치, 탄알이 맞은 표면의 방향을 입력

```
target.OnDamage(gunData.damage, hit.point, hit.normal);
```

15.4 Gun 스크립트(18)

- 레이캐스트 처리가 끝나면 ShotEffect() 코루틴을 실행하고 탄알의 충돌 위치 hitPosition을 입력으로 넘김
 - 탄창에 남은 탄알 수에서 1을 빼기

```
StartCoroutine(ShotEffect(hitPosition));
```

```
magAmmo--;  
if (magAmmo <= 0)  
{  
    state = State.Empty;  
}
```

- 만약 탄창에 탄알이 하나도 남지 않게 되면 총의 현재 상태를 Empty 상태로 변경하고 Shot() 메서드가 종료

15.4 Gun 스크립트(19)

15.4.10 Reload() 메서드

- Reload() 메서드는 재장전을 시도하는 메서드
 - 재장전 실행에 성공하면 true, 재장전을 할 수 없는 상태면 false를 반환
 - 재장전은 대기 시간이 필요하기 때문에 실질적인 재장전은 코루틴 메서드인 ReloadRoutine()에서 수행

[과정 01] Reload () 메서드 완성하기

① Gun 스크립트의 Reload() 메서드를 다음과 같이 완성

```
public bool Reload() {  
    if (state == State.Reloding || ammoRemain <= 0 || magAmmo >= gunData.magCapacity)  
    {  
        // 이미 재장전 중이거나 남은 탄알이 없거나  
        // 탄창에 탄알이 이미 가득한 경우 재장전할 수 없음  
        return false;  
    }  
  
    // 재장전 처리 시작  
    StartCoroutine(ReloadRoutine());  
    return true;  
}
```

Reload() 메서드는 다음과 같은 상황에서는 재장전을 실행하지 않음

- state == state.Reloding : 이미 재장전을 하고 있는 중
- ammoRemain <= 0 : 재장전에 사용할 남은 탄알이 없음
- magAmmo >= gunData.magCapacity : 탄창에 탄알이 이미 가득 차 있음

15.4 Gun 스크립트(20)

15.4.11 ReloadRoutine() 코루틴

- ReloadRoutine() 코루틴 메서드는 탄창에 탄알을 채우고, 재장전 소리를 재생하며, 재장전 시간 동안 총의 다른 기능이 동작하지 않도록 총을 잠금

[과정 01] ReloadRoutine () 코루틴 메서드 완성하기

① Gun 스크립트의 ReloadRoutine() 코루틴 메서드를 다음과 같이 완성

- 현재 상태를 재장전 중인 상태로 변경

```
state = State.Reload;
```

- 오디오 소스로 재장전 오디오 클립을 1회 재생

```
gunAudioPlayer.PlayOneShot(gunData.reloadClip);
```

- 재장전 시간만큼 처리를 쉬

```
yield return new WaitForSeconds(gunData.reloadTime);
```

15.4 Gun 스크립트(21)

- 대기 시간이 끝나면 탄창에 채워 넣어야 할 탄알 수 ammoToFill을 계산
 - 채워 넣을 탄알 수 = 탄창의 최대 용량 - 탄창에 남아 있는 탄알 수

```
int ammoToFill = magCapacity - magAmmo;
```

```
if (ammoRemain < ammoToFill)
{
    ammoToFill = ammoRemain;
}
```

- 채워 넣을 탄알 ammoToFill의 값을 계산한 다음에는 그만큼 탄창에 탄알을 집어넣음
 - 그리고 탄창에 넣은 탄알만큼 전체 탄알을 감소시킴
 - 그다음 총의 현재 상태를 발사 준비된 상태(Ready)로 변경하여 총의 잠금을 해제

```
magAmmo += ammoToFill;
ammoRemain -= ammoToFill;
```

```
state = State.Ready;
```

15.4 Gun 스크립트(22)

15.4.12 완성된 Gun 스크립트

```
using System.Collections;
using UnityEngine;

// 총을 구현
public class Gun : MonoBehaviour {
    // 총의 상태를 표현하는 데 사용할 타입을 선언
    public enum State {
        Ready, // 발사 준비됨
        Empty, // 탄창이 빈
        Reloading // 재장전 중
    }

    public State state { get; private set; } // 현재 총의 상태

    public Transform fireTransform; // 탄알이 발사될 위치

    public ParticleSystem muzzleFlashEffect; // 총구 화염 효과
    public ParticleSystem shellEjectEffect; // 탄피 배출 효과
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(23)

◀ 앞쪽에 이어

```
private LineRenderer bulletLineRenderer; // 탄알 궤적을 그리기 위한 렌더러
```

```
private AudioSource gunAudioPlayer; // 총 소리 재생기
```

```
public GunData gunData; // 총의 현재 데이터
```

```
private float fireDistance = 50f; // 사정거리
```

```
public int ammoRemain = 100; // 남은 전체 탄알
```

```
public int magAmmo; // 현재 탄창에 남아 있는 탄알
```

```
private float lastFireTime; // 총을 마지막으로 발사한 시점
```

```
private void Awake() {
```

```
    // 사용할 컴포넌트의 참조 가져오기
```

```
    gunAudioPlayer = GetComponent();
```

```
    bulletLineRenderer = GetComponent<LineRenderer>();
```

```
    // 사용할 점을 두 개로 변경
```

```
    bulletLineRenderer.positionCount = 2;
```

```
    // 라인 렌더러를 비활성화
```

```
    bulletLineRenderer.enabled = false;
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(24)

◀ 앞쪽에 이어

```
private void OnEnable() {  
    // 전체 예비 탄알 양을 초기화  
    ammoRemain = gunData.startAmmoRemain;  
    // 현재 탄창을 가득 채우기  
    magAmmo = gunData.magCapacity;  
    // 총의 현재 상태를 총을 쏠 준비가 된 상태로 변경  
    state = State.Ready;  
    // 마지막으로 총을 쏜 시점을 초기화  
    lastFireTime = 0;  
}  
  
// 발사 시도  
public void Fire() {  
    // 현재 상태가 발사 가능한 상태  
    // && 마지막 총 발사 시점에서 timeBetFire 이상의 시간이 지남  
    if (state == State.Ready && Time.time >= lastFireTime + gunData.timeBetFire)  
    {  
        // 마지막 총 발사 시점 갱신  
        lastFireTime = Time.time;  
        // 실제 발사 처리 실행  
        Shot();  
    }  
}
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(25)

◀ 앞쪽에 이어

// 실제 발사 처리

```
private void Shot() {
```

// 레이캐스트에 의한 충돌 정보를 저장하는 컨테이너

```
RaycastHit hit;
```

// 탄알이 맞은 곳을 저장할 변수

```
Vector3 hitPosition = Vector3.zero;
```

// 레이캐스트(시작 지점, 방향, 충돌 정보 컨테이너, 사정거리)

```
if (Physics.Raycast(fireTransform.position, fireTransform.forward, out hit,  
    fireDistance))
```

```
{
```

// 레이가 어떤 물체와 충돌한 경우

// 충돌한 상대방으로부터 IDamageable 오브젝트 가져오기 시도

```
IDamageable target = hit.collider.GetComponent<IDamageable>();
```

// 상대방으로부터 IDamageable 오브젝트를 가져오는 데 성공했다면

```
if (target != null)
```

```
{
```

// 상대방의 OnDamage 함수를 실행시켜 상대방에 데미지 주기

```
target.OnDamage(gunData.damage, hit.point, hit.normal);
```

```
}
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(26)

◀ 앞쪽에 이어

```
// 레이가 충돌한 위치 저장
hitPosition = hit.point;
}
else
{
    // 레이가 다른 물체와 충돌하지 않았다면
    // 탄알이 최대 사정거리까지 날아갔을 때의 위치를 충돌 위치로 사용
    hitPosition = fireTransform.position + fireTransform.forward * fireDistance;
}

// 발사 이펙트 재생 시작
StartCoroutine(ShotEffect(hitPosition));

// 남은 탄알 수를 -1
magAmmo--;
if (magAmmo <= 0)
{
    // 탄창에 남은 탄알이 없다면 총의 현재 상태를 Empty로 갱신
    state = State.Empty;
}
}
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(27)

◀ 앞쪽에 이어

```
// 발사 이펙트와 소리를 재생하고 탄알 궤적을 그림
private IEnumerator ShotEffect(Vector3 hitPosition) {
    // 총구 화염 효과 재생
    muzzleFlashEffect.Play();
    // 탄피 배출 효과 재생
    shellEjectEffect.Play();

    // 총격 소리 재생
    gunAudioPlayer.PlayOneShot(gunData.shotClip);

    // 선의 시작점은 총구의 위치
    bulletLineRenderer.SetPosition(0, fireTransform.position);
    // 선의 끝점은 입력으로 들어온 충돌 위치
    bulletLineRenderer.SetPosition(1, hitPosition);
    // 라인 렌더러를 활성화하여 탄알 궤적을 그림
    bulletLineRenderer.enabled = true;

    // 0.03초 동안 잠시 처리를 대기
    yield return new WaitForSeconds(0.03f);

    // 라인 렌더러를 비활성화하여 탄알 궤적을 지움
    bulletLineRenderer.enabled = false;
}
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(28)

◀ 앞쪽에 이어

// 재장전 시도

```
public bool Reload() {  
    if (state == State.Reloding || ammoRemain <= 0 || magAmmo >= gunData.magCapacity)  
    {  
        // 이미 재장전 중이거나 남은 탄알이 없거나  
        // 탄창에 탄알이 이미 가득 찬 경우 재장전할 수 없음  
        return false;  
    }  
  
    // 재장전 처리 시작  
    StartCoroutine(ReloadRoutine());  
    return true;  
}
```

// 실제 재장전 처리를 진행

```
private IEnumerator ReloadRoutine() {  
    // 현재 상태를 재장전 중 상태로 전환  
    state = State.Reloding;  
    // 재장전 소리 재생  
    gunAudioPlayer.PlayOneShot(gunData.reloadClip);  
  
    // 재장전 소요 시간만큼 처리 쉬기  
    yield return new WaitForSeconds(gunData.reloadTime);
```

▶ 다음 쪽에 이어짐

15.4 Gun 스크립트(29)

◀ 앞쪽에 이어

```
// 탄창에 채울 탄알 계산
int ammoToFill = gunData.magCapacity - magAmmo;

// 탄창에 채워야 할 탄알이 남은 탄알보다 많다면
// 채워야 할 탄알 수를 남은 탄알 수에 맞춰 줄임
if (ammoRemain < ammoToFill)
{
    ammoToFill = ammoRemain;
}

// 탄창을 채움
magAmmo += ammoToFill;
// 남은 탄알에서 탄창에 채운만큼 탄알을 뺌
ammoRemain -= ammoToFill;
// 총의 현재 상태를 발사 준비된 상태로 변경
state = State.Ready;
}
}
```

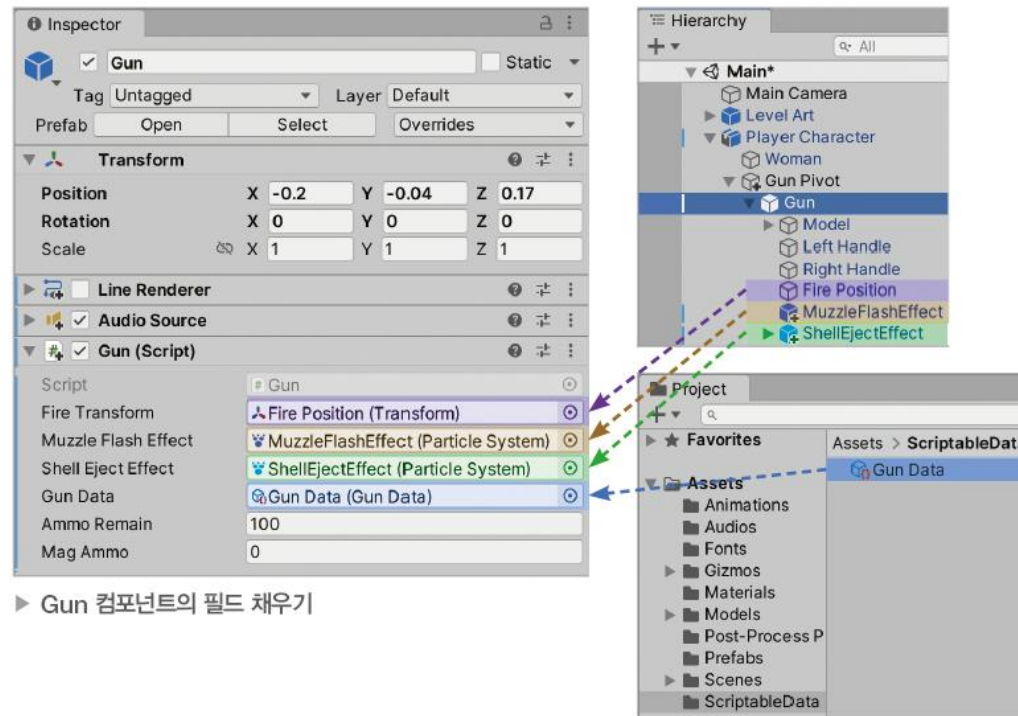
15.4 Gun 스크립트(30)

15.4.13 Gun 컴포넌트 설정

- 완성된 Gun 컴포넌트의 필드를 채워 Gun 게임 오브젝트를 완성

[과정 01] Gun 컴포넌트의 필드 채우기

- ① Fire Position 게임 오브젝트를 Fire Transform 필드로 드래그&드롭
- ② MuzzleFlashEffect 게임 오브젝트를 Muzzle Flash Effect 필드로 드래그&드롭
- ③ ShellEjectEffect 게임 오브젝트를 Shell Eject Effect 필드로 드래그&드롭
- ④ Gun Data 스크립터블 오브젝트 에셋을 Gun Data 필드에 할당
(필드 옆 선택 버튼 사용 또는 ScriptableData 폴더의 Gun Data를 드래그&드롭)

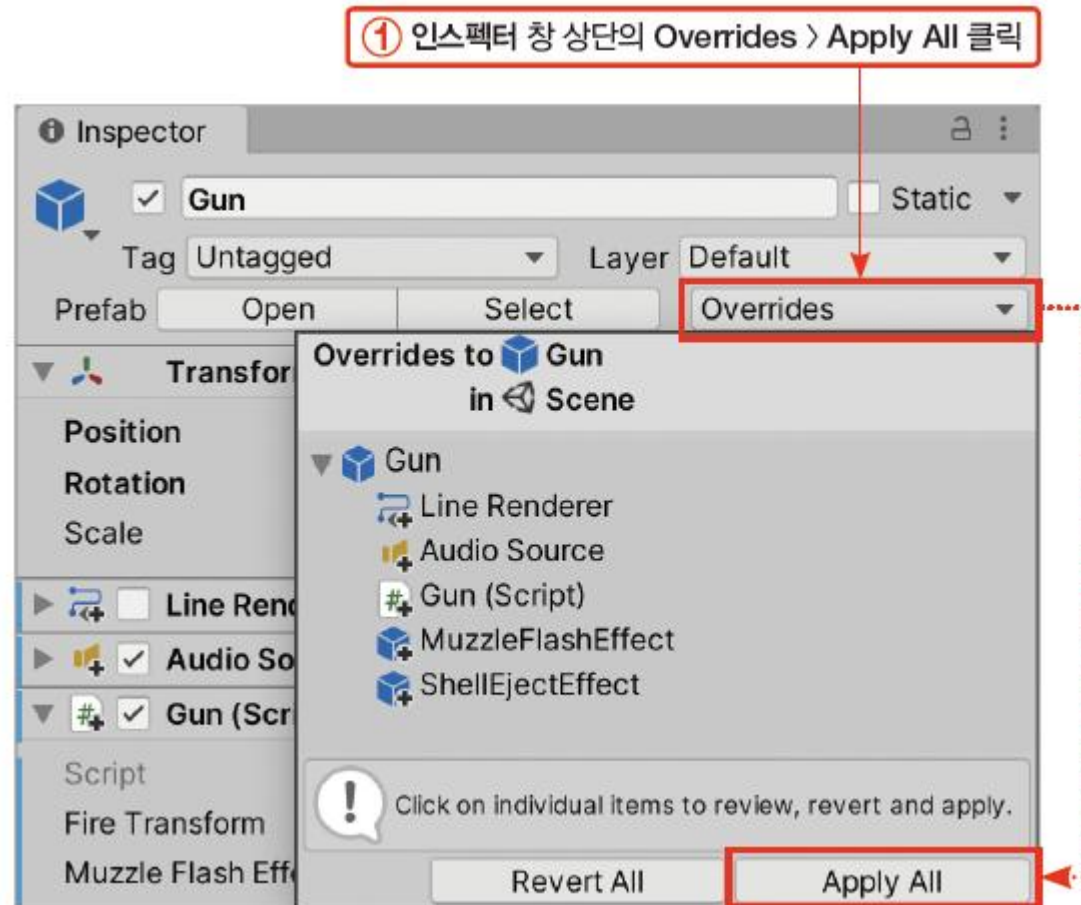


15.4 Gun 스크립트(31)

- Gun 프리팹에 변경 사항을 모두 반영하여 Gun 게임 오브젝트와 프리팹을 완성

[과정 02] Gun 프리팹 갱신하기

- ① 인스펙터 창 상단의 Overrides > Apply All 클릭



15.5 슈터 만들기(1)

15.5.1 IK

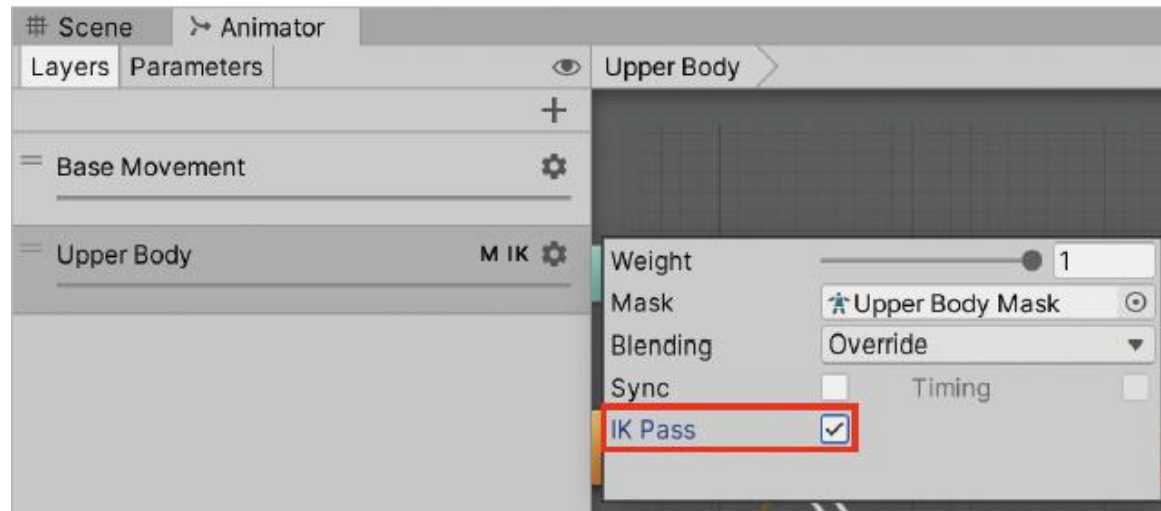
FK

- 캐릭터 애니메이션은 기본적으로 FK Forward Kinematics (전진 운동학)로 동작
- FK에서는 부모 조인트joint에서 자식 조인트 순서로 움직임을 적용
- FK로 물건을 잡는 애니메이션을 재생한다고 가정
 1. 어깨를 움직입니다. 어깨에 종속된 팔이 같이 움직임
 2. 팔을 움직입니다. 팔에 종속된 손이 같이 움직임
 3. 손을 움직임
- FK에서 손의 위치는 순서대로 계산된 최종 결과
 - FK로 물건을 잡는 애니메이션을 재생하면 물건의 위치에 맞춰 손의 위치를 변형할 수 없기 때문에 손의 위치로 물건을 순간이동

15.5 슈터 만들기(2)

IK

- IK(Inverse Kinematics, 역운동학)는 자식 조인트의 위치를 먼저 결정하고 부모 조인트가 거기에 맞춰 변형
- IK로 물건을 집는 애니메이션을 재생한다고 가정
 1. 손의 위치를 물건의 위치로 이동
 2. 팔이 손의 위치에 맞춰 움직임
 3. 어깨가 팔의 위치에 맞춰 움직임
- IK는 하위 조인트의 최종 위치를 먼저 결정할 수 있음
 - 따라서 물건의 위치에 맞춰 손의 위치를 먼저 결정하고, 거기에 맞춰 팔과 어깨의 위치를 결정



▶ 활성화되어 있는 IK Pass

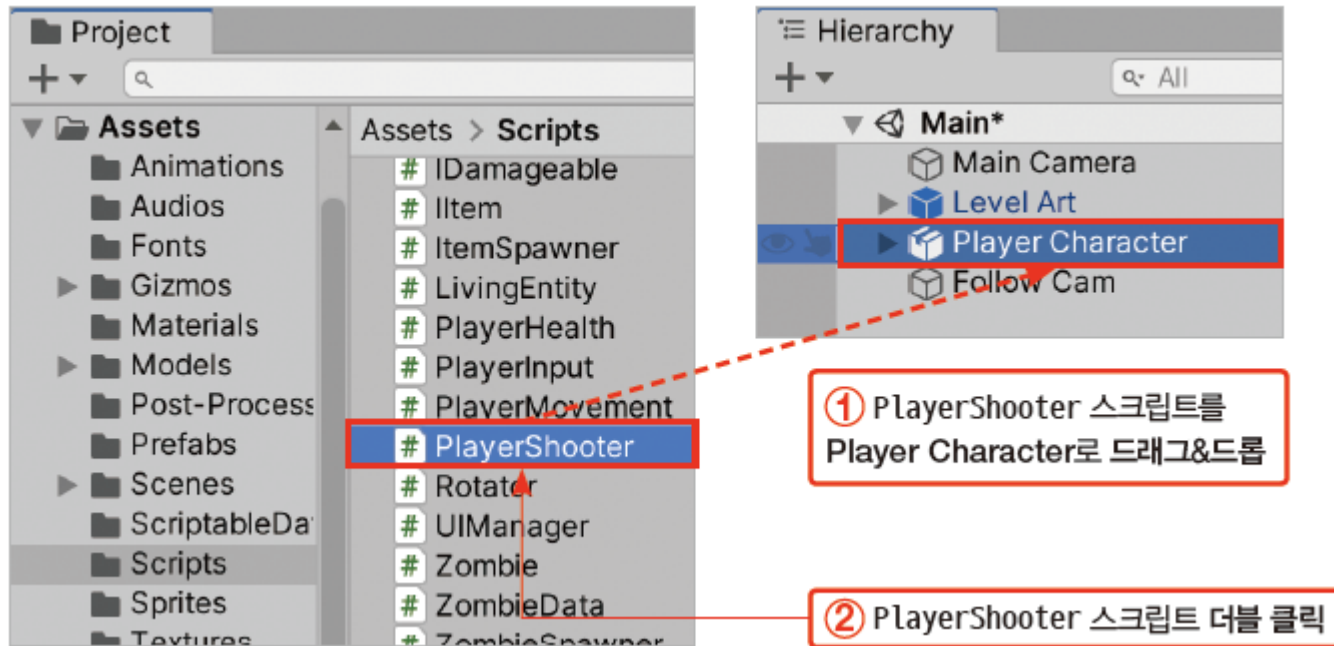
15.5 슈터 만들기(3)

15.5.2 PlayerShooter 스크립트

- PlayerShooter 스크립트를 플레이어 캐릭터에 추가

[과정 01] PlayerShooter 스크립트 추가하기

- ① Scripts 폴더의 PlayerShooter 스크립트를 Player Character 게임 오브젝트로 드래그&드롭
- ② PlayerShooter 스크립트를 더블 클릭으로 열기



(코드 생략)

15.5 슈터 만들기(4)

- gun은 사용할 Gun 게임 오브젝트의 Gun 컴포넌트

```
public Gun gun;
```

- IK 갱신에 사용할 변수들이 선언

```
public Transform gunPivot;  
public Transform leftHandMount;  
public Transform rightHandMount;
```

- gunPivot은 Gun 게임 오브젝트를 배치하는 기준으로 사용할 트랜스폼
- leftHandMount와 rightHandMount는 총의 왼쪽 손잡이와 오른쪽 손잡이 위치를 표시할 트랜스폼 컴포넌트

- PlayerCharacter 게임 오브젝트의 컴포넌트에 대한 변수가 선언

```
private PlayerInput playerInput;  
private Animator playerAnimator;
```

- playerInput은 플레이어 입력을 전달하는 PlayerInput 컴포넌트가 할당
- playerAnimator는 플레이어 캐릭터의 애니메이션을 재생하는 애니메이터 컴포넌트가 할당

15.5 슈터 만들기(5)

15.5.3 Start() 메서드

- PlayerShooter 스크립트의 Start() 메서드에서는 사용할 애니메이터 컴포넌트와 PlayerInput 컴포넌트에 대한 참조를 Player Character 게임 오브젝트로부터 가져옴

```
private void Start() {  
    // 사용할 컴포넌트 가져오기  
    playerInput = GetComponent<PlayerInput>();  
    playerAnimator = GetComponent<Animator>();  
}
```

15.5 슈터 만들기(6)

15.5.4 OnEnable(), OnDisable() 메서드

- OnEnable() 메서드는 PlayerShooter 컴포넌트가 활성화될 때 자동으로 실행
- 총을 쏘는 슈터가 활성화되면 총도 함께 활성화되어야 하므로 OnEnable() 메서드에는 총 게임 오브젝트를 활성화하는 처리를 구현

```
private void OnEnable() {  
    // 슈터가 활성화될 때 총도 함께 활성화  
    gun.gameObject.SetActive(true);  
}
```

- 총을 쏘는 슈터가 비활성화되면 총도 함께 비활성화되어야 하므로 OnDisable() 메서드에는 총 게임 오브젝트를 비활성화하는 처리를 구현

```
private void OnDisable() {  
    // 슈터가 비활성화될 때 총도 함께 비활성화  
    gun.gameObject.SetActive(false);  
}
```

15.5 슈터 만들기(7)

15.5.5 Update() 메서드

- PlayerShooter 스크립트의 Update() 메서드에서는 매 프레임마다 플레이어 입력을 감지하고 총을 발사하거나 재장전

[과정 01] Update () 메서드 완성하기

① PlayerShooter 스크립트의 Update() 메서드를 다음과 같이 완성

- 재장전을 시도한 경우 재장전에 성공한 경우에만 재장전 애니메이션을 재생
 - 따라서 gun.Reload()로 재장전을 시도한 결과를 if 문으로 검사하여 재장전에 성공한 경우에만 애니메이터의 Reload 트리거를 발동

```
if (gun.Reload())  
{  
    playerAnimator.SetTrigger("Reload");  
}
```

- Update() 메서드의 마지막 줄에서는 UpdateUI() 메서드를 실행하여 매 프레임마다 탄알 UI를 갱신

15.5 슈터 만들기(8)

15.5.6 UpdateUI() 메서드

- UpdateUI() 메서드는 남은 탄알 UI를 갱신

```
private void UpdateUI() {  
    if (gun != null && UIManager.instance != null)  
    {  
        // UI 매니저의 탄알 텍스트에 탄창의 탄알과 남은 전체 탄알 표시  
        UIManager.instance.UpdateAmmoText(gun.magAmmo, gun.ammoRemain);  
    }  
}
```

15.5 슈터 만들기(9)

15.5.7 OnAnimatorIK() 메서드

- OnAnimatorIK() 메서드의 역할
 - 총을 상체와 함께 흔들기
 - 캐릭터의 양손을 총의 양쪽 손잡이에 위치시키기

[과정 01] OnAnimatorIK () 메서드 완성하기

① **PlayerShooter** 스크립트의 **OnAnimatorIK()** 메서드를 다음과 같이 완성

- 캐릭터의 오른쪽 팔꿈치 위치를 찾아 `unPivot`의 위치로 사용

```
gunPivot.position = playerAnimator.GetIKHintPosition(AvatarIKHint.RightElbow);
```

- AvatarIKHint 타입은 IK 대상 중 다음 부위를 표현하는 타입
 - AvatarIKHint.LeftElbow : 왼쪽 팔꿈치
 - AvatarIKHint.RightElbow : 오른쪽 팔꿈치
 - AvatarIKHint.LeftKnee : 왼쪽 무릎
 - AvatarIKHint.RightKnee : 오른쪽 무릎

15.5 슈터 만들기(10)

- 캐릭터 왼손의 위치와 회전을 leftHandMount의 위치와 회전으로 변경
 - 먼저 왼손 IK에 대한 위치와 회전 가중치를 1.0(100%)으로 변경

```
playerAnimator.SetIKPositionWeight(AvatarIKGoal.LeftHand, 1.0f);  
playerAnimator.SetIKRotationWeight(AvatarIKGoal.LeftHand, 1.0f);
```

- 왼손 IK의 목표 위치와 목표 회전을 leftHandMount의 위치와 회전으로 지정

```
playerAnimator.SetIKPosition(AvatarIKGoal.LeftHand, leftHandMount.position);  
playerAnimator.SetIKRotation(AvatarIKGoal.LeftHand, leftHandMount.rotation);
```

- 오른손은 AvatarIKGoal.RightHand를 대상으로 하여 같은 방식으로 IK를 적용

```
playerAnimator.SetIKPositionWeight(AvatarIKGoal.RightHand, 1.0f);  
playerAnimator.SetIKRotationWeight(AvatarIKGoal.RightHand, 1.0f);  
  
playerAnimator.SetIKPosition(AvatarIKGoal.RightHand, rightHandMount.position);  
playerAnimator.SetIKRotation(AvatarIKGoal.RightHand, rightHandMount.rotation);
```

15.5 슈터 만들기(11)

15.5.8 완성된 PlayerShooter 스크립트

```
using UnityEngine;

// 주어진 Gun 오브젝트를 쏘거나 재장전
// 알맞은 애니메이션을 재생하고 IK를 사용해 캐릭터 양손이 총에 위치하도록 조정
public class PlayerShooter : MonoBehaviour {
    public Gun gun; // 사용할 총
    public Transform gunPivot; // 총 배치의 기준점
    public Transform leftHandMount; // 총의 왼쪽 손잡이, 왼손이 위치할 지점
    public Transform rightHandMount; // 총의 오른쪽 손잡이, 오른손이 위치할 지점

    private PlayerInput playerInput; // 플레이어의 입력
    private Animator playerAnimator; // 애니메이터 컴포넌트

    private void Start() {
        // 사용할 컴포넌트 가져오기
        playerInput = GetComponent<PlayerInput>();
        playerAnimator = GetComponent<Animator>();
    }
}
```

▶ 다음 쪽에 이어짐

15.5 슈터 만들기(12)

◀ 앞쪽에 이어

```
private void OnEnable() {  
    // 슈터가 활성화될 때 총도 함께 활성화  
    gun.gameObject.SetActive(true);  
}
```



```
private void OnDisable() {  
    // 슈터가 비활성화될 때 총도 함께 비활성화  
    gun.gameObject.SetActive(false);  
}
```

```
private void Update() {  
    // 입력을 감지하고 총 발사하거나 재장전  
    if (playerInput.fire)  
    {  
        // 발사 입력 감지시 총 발사  
        gun.Fire();  
    }  
    else if (playerInput.reload)  
    {  
        // 재장전 입력 감지시 재장전  
        if (gun.Reload())  
        {  
            // 재장전 성공시에만 재장전 애니메이션 재생  
            playerAnimator.SetTrigger("Reload");  
        }  
    }  
  
    // 남은 탄약 UI 갱신  
    UpdateUI();  
}
```

▶ 다음 쪽에 이어짐

15.5 슈터 만들기(13)

◀ 앞쪽에 이어

```
// 탄알 UI 갱신
private void UpdateUI() {
    if (gun != null && UIManager.instance != null)
    {
        // UI 매니저의 탄알 텍스트에 탄창의 탄알과 남은 전체 탄알 표시
        UIManager.instance.UpdateAmmoText(gun.magAmmo, gun.ammoRemain);
    }
}
```

▶ 다음 쪽에 이어짐

15.5 슈터 만들기(14)

◀ 앞쪽에 이어

// 애니메이터의 IK 갱신

```
private void OnAnimatorIK(int layerIndex) {
```

// 총의 기준점 gunPivot을 3D 모델의 오른쪽 팔꿈치 위치로 이동

```
gunPivot.position = playerAnimator.GetIKHintPosition(AvatarIKHint.RightElbow);
```

// IK를 사용하여 왼손의 위치와 회전을 총의 왼쪽 손잡이에 맞춤

```
playerAnimator.SetIKPositionWeight(AvatarIKGoal.LeftHand, 1.0f);
```

```
playerAnimator.SetIKRotationWeight(AvatarIKGoal.LeftHand, 1.0f);
```

```
playerAnimator.SetIKPosition(AvatarIKGoal.LeftHand, leftHandMount.position);
```

```
playerAnimator.SetIKRotation(AvatarIKGoal.LeftHand, leftHandMount.rotation);
```

// IK를 사용하여 오른손의 위치와 회전을 총의 오른쪽 손잡이에 맞춤

```
playerAnimator.SetIKPositionWeight(AvatarIKGoal.RightHand, 1.0f);
```

```
playerAnimator.SetIKRotationWeight(AvatarIKGoal.RightHand, 1.0f);
```

```
playerAnimator.SetIKPosition(AvatarIKGoal.RightHand, rightHandMount.position);
```

```
playerAnimator.SetIKRotation(AvatarIKGoal.RightHand, rightHandMount.rotation);
```

```
}
```

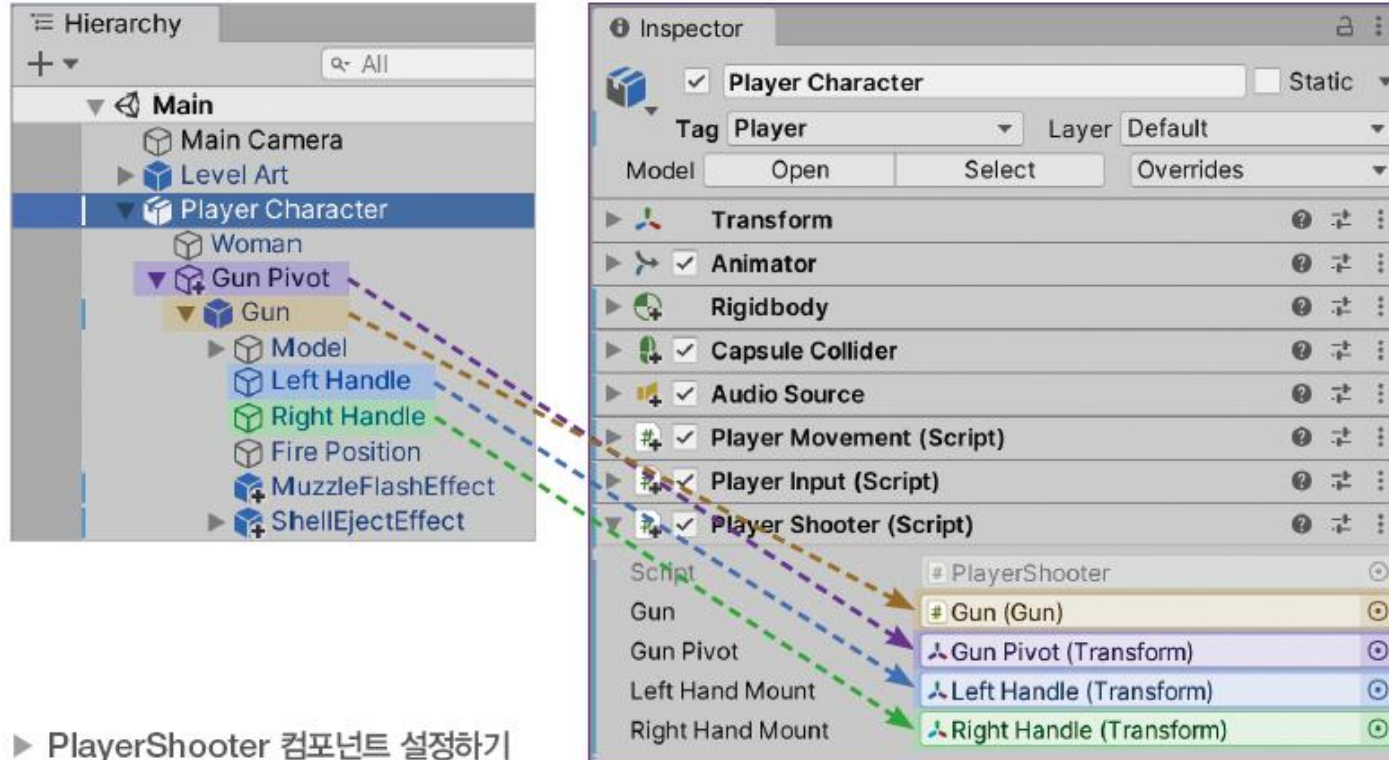
```
}
```

15.5 슈터 만들기(15)

15.5.9 PlayerShooter 컴포넌트 설정하기

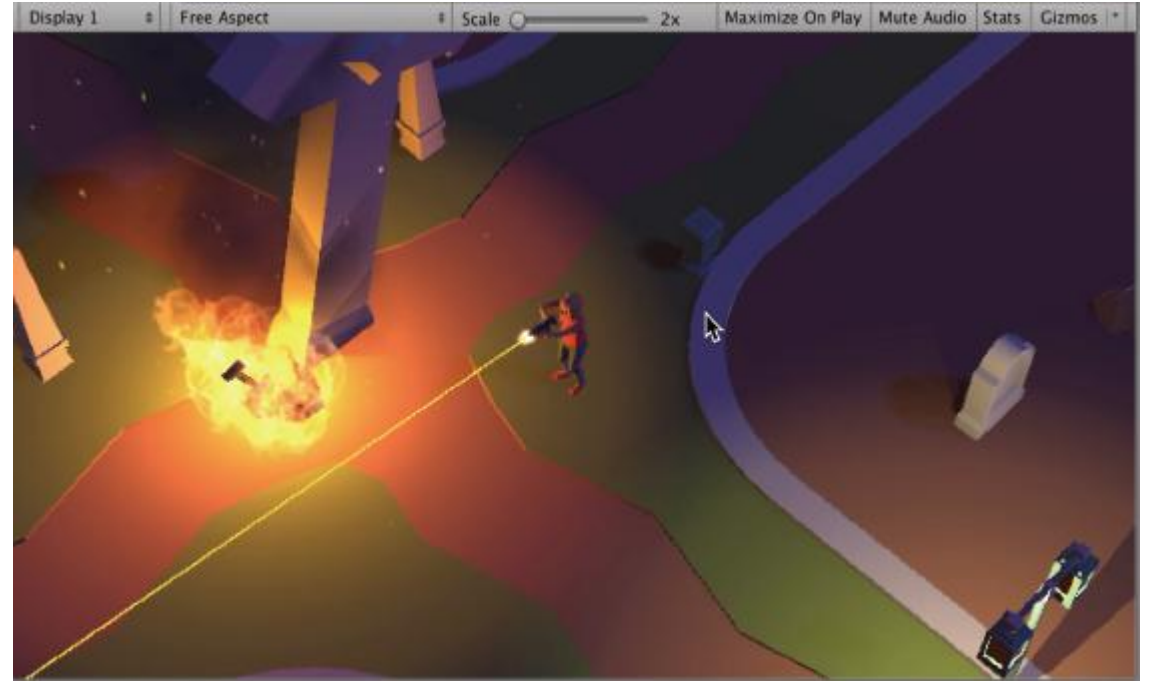
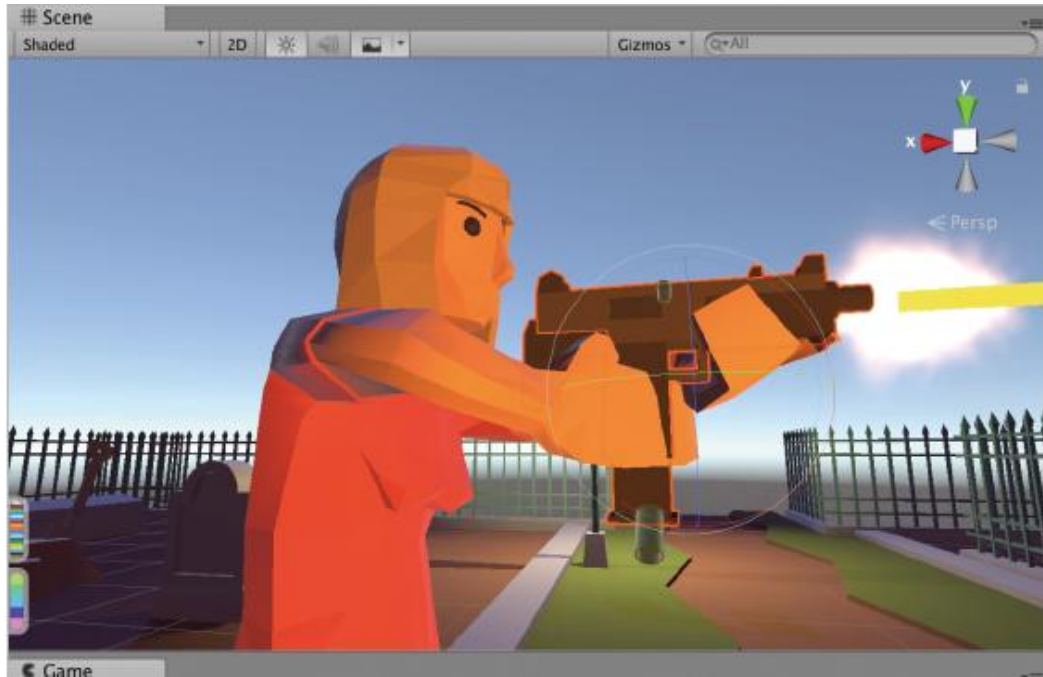
[과정 01] PlayerShooter 컴포넌트 설정하기

- ① Gun 게임 오브젝트를 Gun 필드로 드래그&드롭
- ② Gun Pivot 게임 오브젝트를 Gun Pivot 필드로 드래그&드롭
- ③ Left Handle 게임 오브젝트를 Left Hand Mount 필드로 드래그&드롭
- ④ Right Handle 게임 오브젝트를 Right Hand Mount 필드로 드래그&드롭



15.5 슈터 만들기(16)

- 완성된 슈터 시스템 테스트



15.6 마치며

이 장에서 배운 내용 요약

- 인터페이스는 어떤 메서드를 반드시 구현한다는 계약
- 인터페이스를 상속한 클래스는 반드시 인터페이스의 메서드를 public으로 구현
- 여러 타입을 하나의 인터페이스로 다룰 수 있음
- 스크립터블 오브젝트를 통해 데이터를 유니티 에셋 형태로 저장
- 특수 시각 효과는 파티클 시스템 컴포넌트로 만들
- 코루틴 메서드는 처리 중간에 대기 시간을 삽입할 수 있음
- 코루틴 메서드는 IEnumerator 타입을 반환
- 코루틴에서 대기 시간은 yield return new WaitForSeconds(시간);으로 지정
- 코루틴은 StartCoroutine() 메서드로 실행
- 레이캐스트는 광선을 쏘서 충돌하는 콜라이더가 있는지 검사하는 방법
- 레이캐스트는 Physics.Raycast() 메서드로 실행
- 레이캐스트를 실행하면 충돌 결과 정보가 RaycastHit 타입으로 생성
- IK는 하위 조인트를 먼저 결정하고, 그것에 상위 조인트를 맞추는 애니메이션 방식
- IK 적용은 스크립트에서 OnAnimatorIK() 메서드로 구현
- IK 가중치는 SetIKPositionWeight()와 SetIKRotationWeight()로 결정
- IK 대상의 위치와 회전은 SetIKPosition()과 SetIKRotation()으로 결정
- IK 대상은 AvatarIKGoal 타입으로 나타낼 수 있음